Learning to generate

Distributions, autoencoders and VAEs, built from one handwritten 3



A 3 turns into a 7. Every frame is decoded from a point on a straight line through the code space of the VAE we train today.

Nipun Batra · IIT Gandhinagar

START WITH A QUESTION

Who are these people?

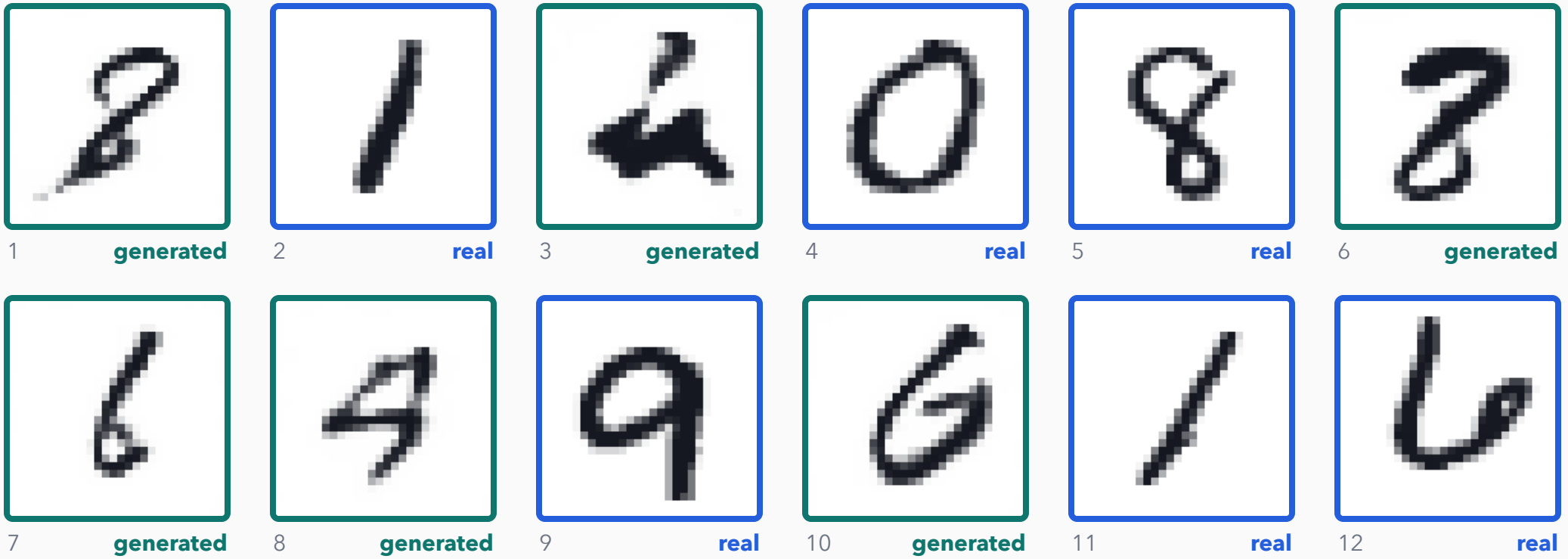


No one. Each face is a random vector of 512 numbers pushed through a trained network.

Karras, Laine & Aila, *A Style-Based Generator Architecture for GANs* (StyleGAN), 2019 · CC BY-NC 4.0

YOUR TURN TO BE THE JUDGE

Real or generated?



Generated: 1, 3, 6, 7, 8, 10 (teal frames). Six from a small GAN, trained in about 9 minutes: the topic of the next lecture.

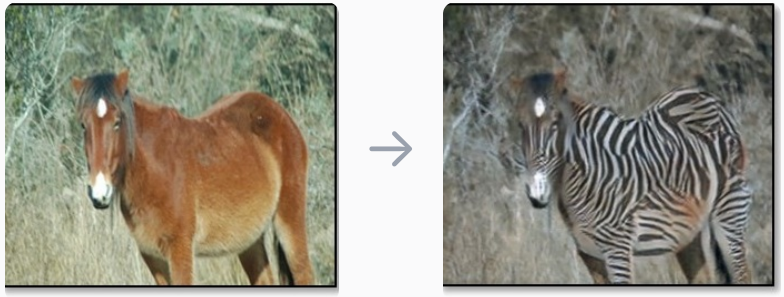
GENERATION IS NOT ONLY "MAKE A RANDOM PICTURE"

Predict the output, then look

a sketch → a photo



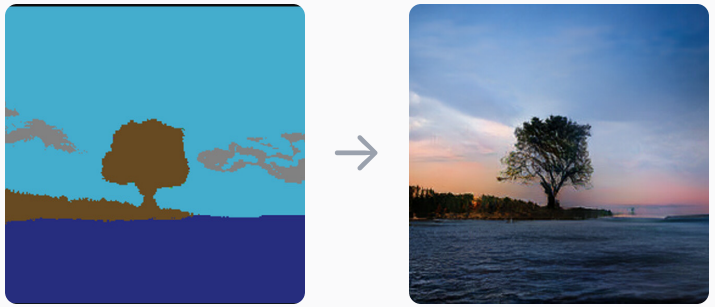
a horse → a zebra



fill a hole



a label map → a landscape



> Each input allows many plausible outputs. A generator must pick one that looks real.

pix2pix (Isola et al. 2017) · CycleGAN (Zhu et al. 2017) · Context Encoders (Pathak et al. 2016) · SPADE (Park et al. 2019). All published model outputs.

ONE INPUT, MANY ANSWERS

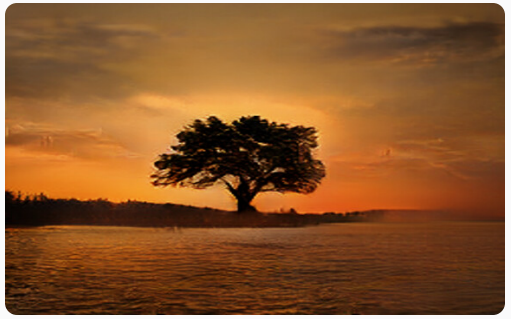
The same label map, three different photographs



the condition



sample 1



sample 2



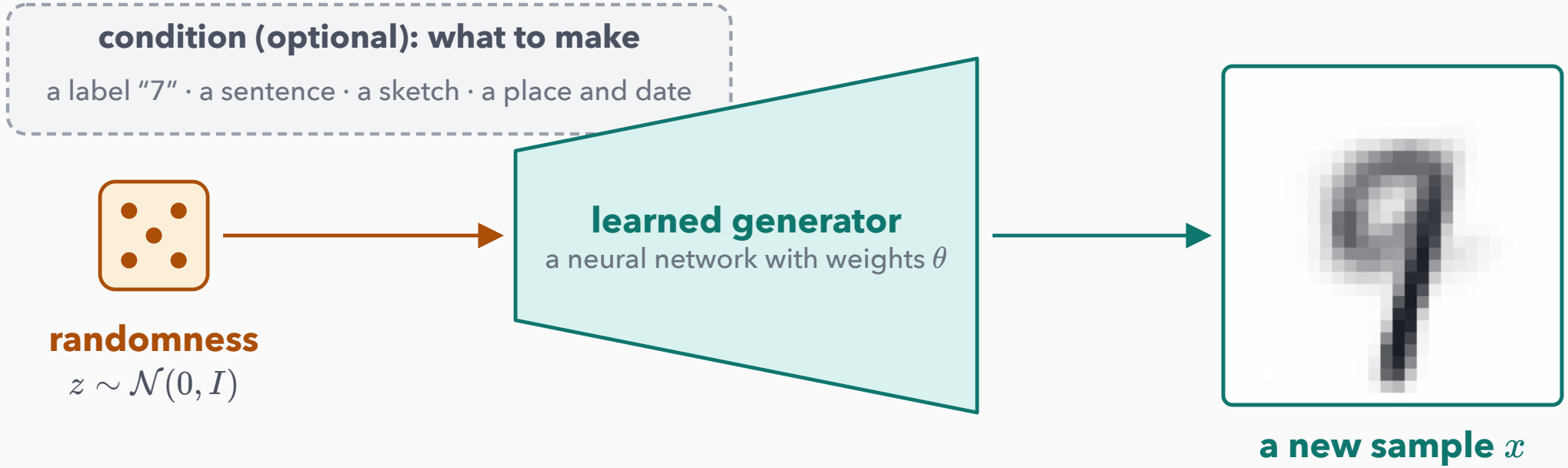
sample 3

A generator does not learn one answer. It learns a distribution of answers and draws from it.

SPADE / GauGAN, Park et al. 2019, teaser figure: here the variation comes from a style image given to the generator.

WHAT ALL OF THESE SHARE

Randomness in, a learned network, a sample out



Training means: adjust θ until the samples look like the data. But what is the loss?

WHAT CAN THE CONDITION BE?

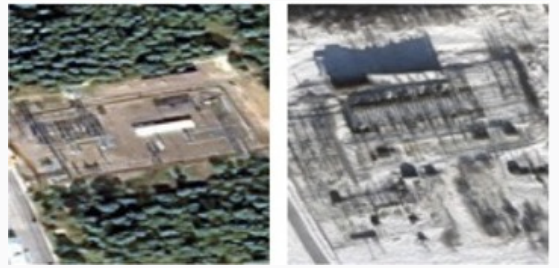
Tell the generator what you want

a label $c = 7$



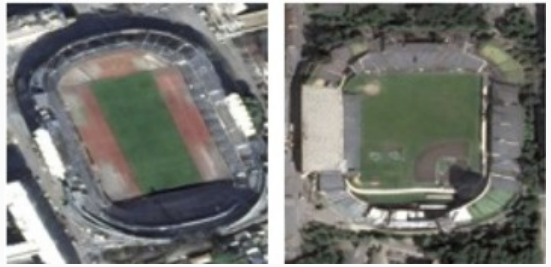
our own conditional VAE: eight different random z, all asked for a 7

a sentence + a date "`a satellite image of an electric substation in Finland`", August vs January



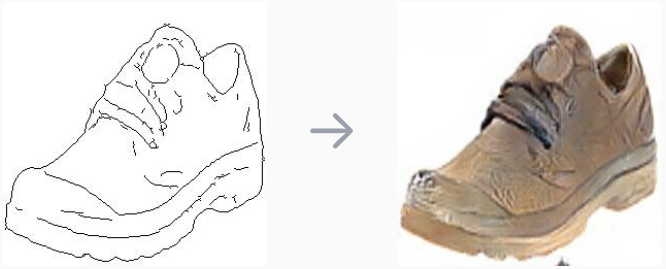
DiffusionSat (Khanna et al., 2024): same caption, only the month changes

a sentence + a place "`a satellite image of a stadium`", France vs USA coordinates



DiffusionSat: same caption, only the latitude and longitude change

an image a sketch, a label map, a photo with a hole



pix2pix, from the earlier slide

> **The condition fixes what; the randomness still decides which one of the many matching outputs you get.**

OUR CONDITIONAL VAE, ALL LABELS AT ONCE

The label picks the digit; the random z picks the style



Each **row**: one label c, from 0 to 9.

Each **column**: one random z, the same all the way down.

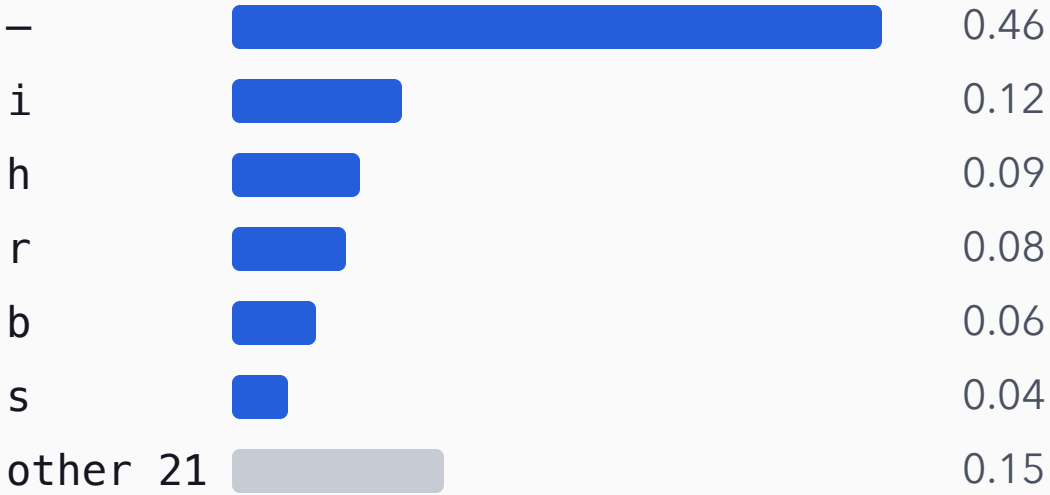
Read down a column: every digit is written with the same slant and thickness. The code z carries the handwriting; the label carries the identity.

EARLIER IN THIS COURSE

You have already built a generator

Our name model reads three characters and gives a probability for every next character.

after `a a b`, the trained model says:



Generate: sample a character, append it, repeat.



A different random draw at any step gives a different name. That is a generative model: it gives probabilities, and we sample from them.

Same idea today: learn a distribution, then sample from it. Only now the samples are images.

EARLIER IN THIS COURSE

A whole name gets one probability

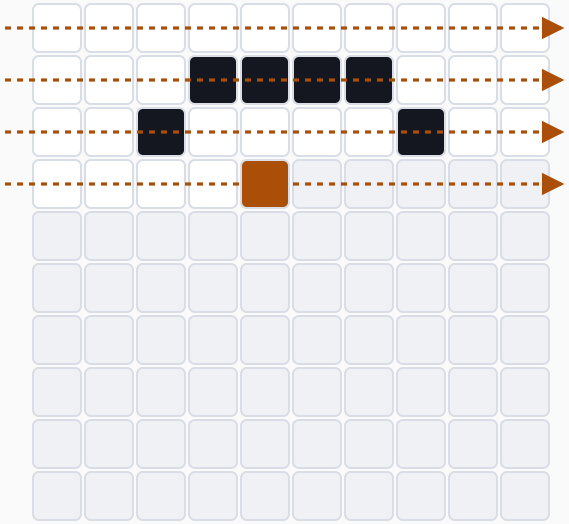
context	next	probability	−log p
`– – –`	`a`	0.08	2.53
`– – a`	`a`	0.08	2.53
`– a a`	`b`	0.02	3.91
`a a b`	`i`	0.12	2.12
`a b i`	`d`	0.09	2.41
`b i d`	`–`	0.23	1.47
aabid		**3.2×10^{-7}**	**14.96**

$$p_\theta(\text{aabid}) = \prod_t p_\theta(x_t \mid x_{<t}) \qquad -\log p_\theta(\text{aabid}) = \sum_t -\log p_\theta(x_t \mid x_{<t})$$

> **Training made this number large for every name in the data. That is maximum likelihood, and it is what we want for images too.**

Two ways to generate a 28 × 28 digit

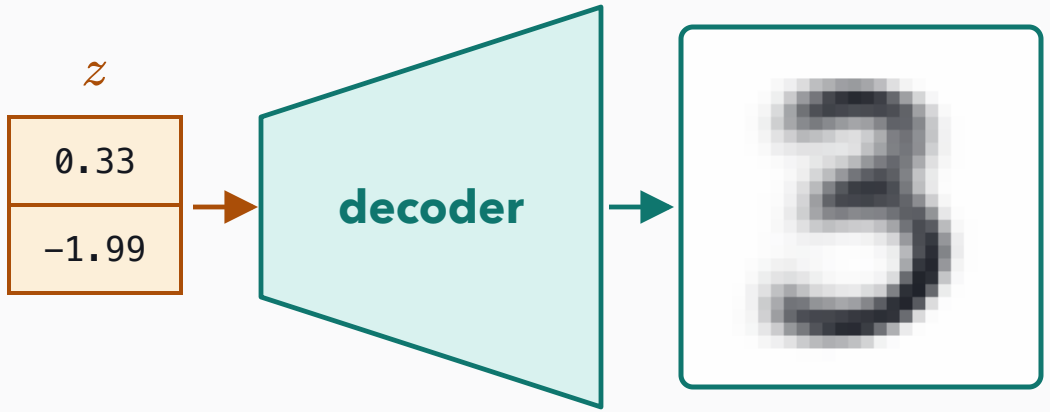
A · one pixel at a time



784 predictions in a row, like our names

$$p(x) = \prod_i p(x_i \mid x_{<i})$$

B · all pixels from one random vector



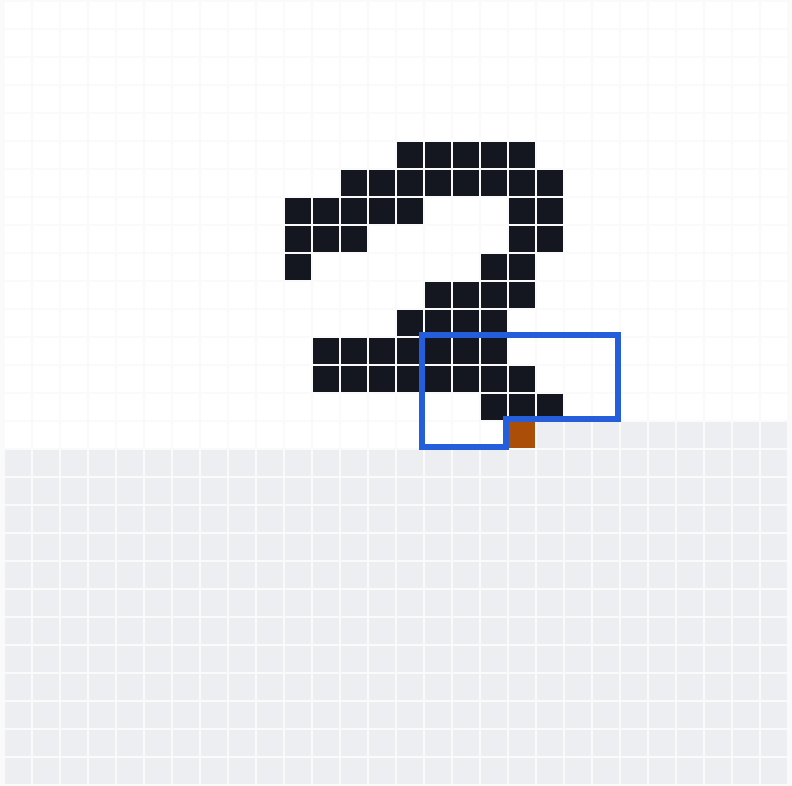
one pass: 2 numbers in, 784 pixels out

$$x = \text{decoder}(z)$$

Today: route B. Two questions: where does z come from, and how do we train a decoder when no one tells us the right z?

ROUTE A, THE WAY OUR NAME MODEL WORKED

Predict each pixel from the pixels already drawn

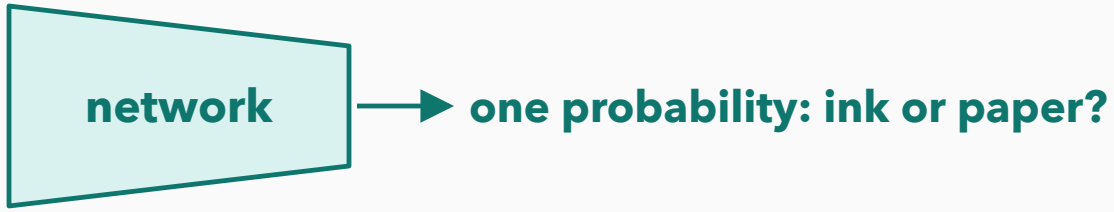


drawn so far · next pixel (orange) · not yet drawn (grey)

context: 3 rows above + 3 to the left

24 pixels the model may look at

$$p(x_i = \text{ink} \mid 24 \text{ pixels}) = 0.89$$



Repeat for all 784 pixels, one after another:

$$p(x) = \prod_{i=1}^{784} p(x_i \mid x_{<i})$$

ROUTE A, TRAINED ON MNIST

What these two pixel-by-pixel models generate



context: 3 previous pixels · $\log p(\text{our } 3) = -123.7$



context: 24 pixels above and left · $\log p(\text{our } 3) = -69.4$

More context, better strokes. And route A gives an exact $\log p(x)$: just add the 784 log-probabilities.

SO WHY ROUTE B TODAY?

Two routes, different strengths

	A · pixel by pixel	B · one code, one pass
generate one image	784 network calls, in order	1 network call
$\log p(x)$ **of an image**	**exact**: a sum of 784 terms	an integral over codes: hard
a short summary z	none	**yes**: 2 to 100 numbers you can move, mix, edit
used today in	language models; image models over image tokens	VAEs, GANs, and the latent space of diffusion models

> **Route A is the name model you already know. Today is route B: what the code is, and how to train a decoder when nobody tells us the right code.**

THE PLAN

Today's route, in two sessions

SESSION 1

1. **Distributions and fitting**
   a 1-D toy: fit, sample, how many bumps
2. **Autoencoders**
   plain, denoising, sparse, contractive; why they cannot generate
3. **The probabilistic view**
   prior, likelihood, an intractable posterior

SESSION 2

4. **KL divergence**
   how far apart are two distributions?
5. **Variational inference and the ELBO**
   a loss that never computes $p(x)$
6. **Reparameterisation**
   backprop through a random draw
7. **The VAE on MNIST**
   one digit, end to end, with real numbers

> **One running image throughout: test digit #90, a handwritten 3. GANs get their own lecture.**

PART 1 OF 7

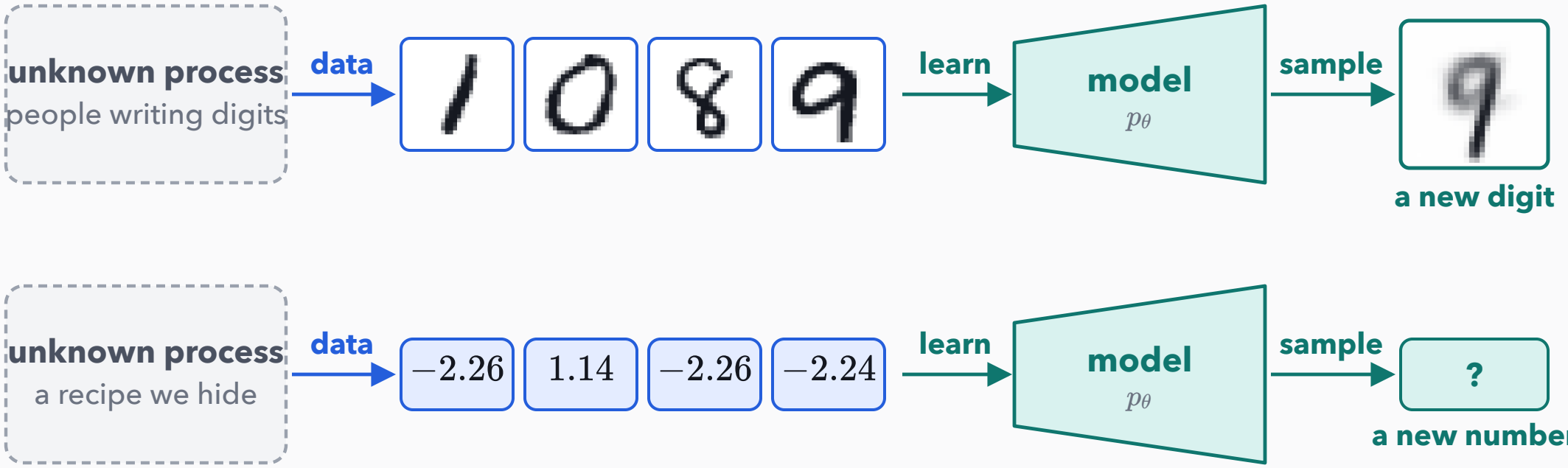
Learn a distribution from examples, then sample from it

We have seen what generators can do. Before any network, the whole idea on **one number**: what does "learning" mean, and how do we then make new samples?

1 **Distributions and fitting**
2 Autoencoders
3 The probabilistic view
4 KL divergence
5 Variational inference and the ELBO
6 Reparameterisation
7 The VAE on MNIST

WHAT ARE WE TRYING TO DO?

Examples in, a model out, new examples from the model



1 · Learn

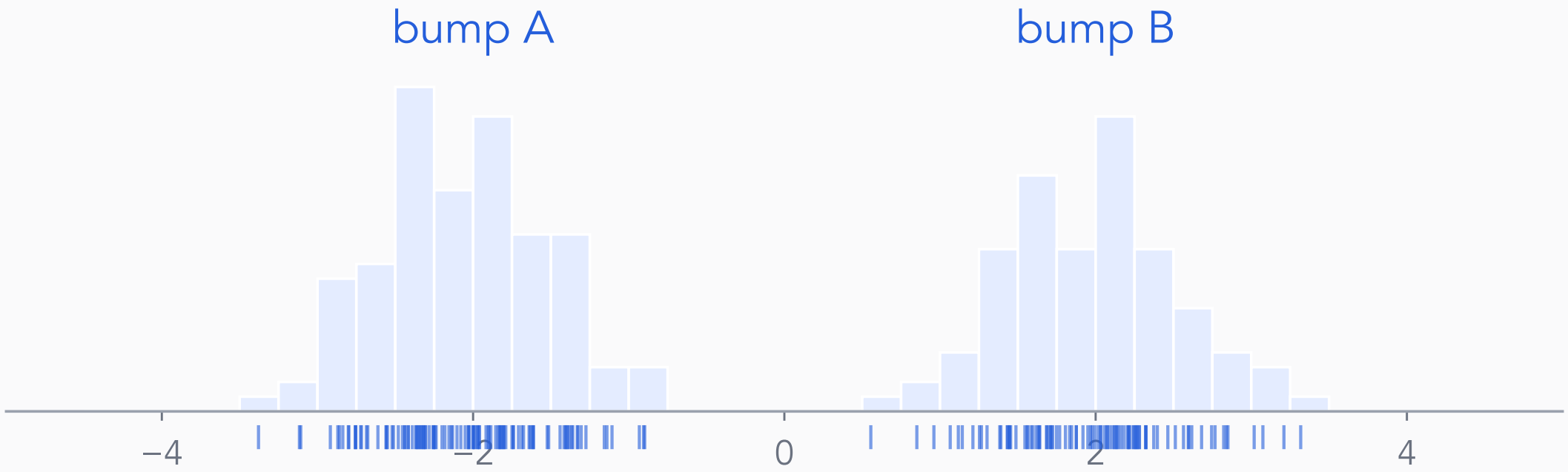
Choose the model's parameters θ so that the examples we have are **probable** under it.

2 · Generate

Draw **new** samples from the learned model p_θ.

OUR TOY DATA

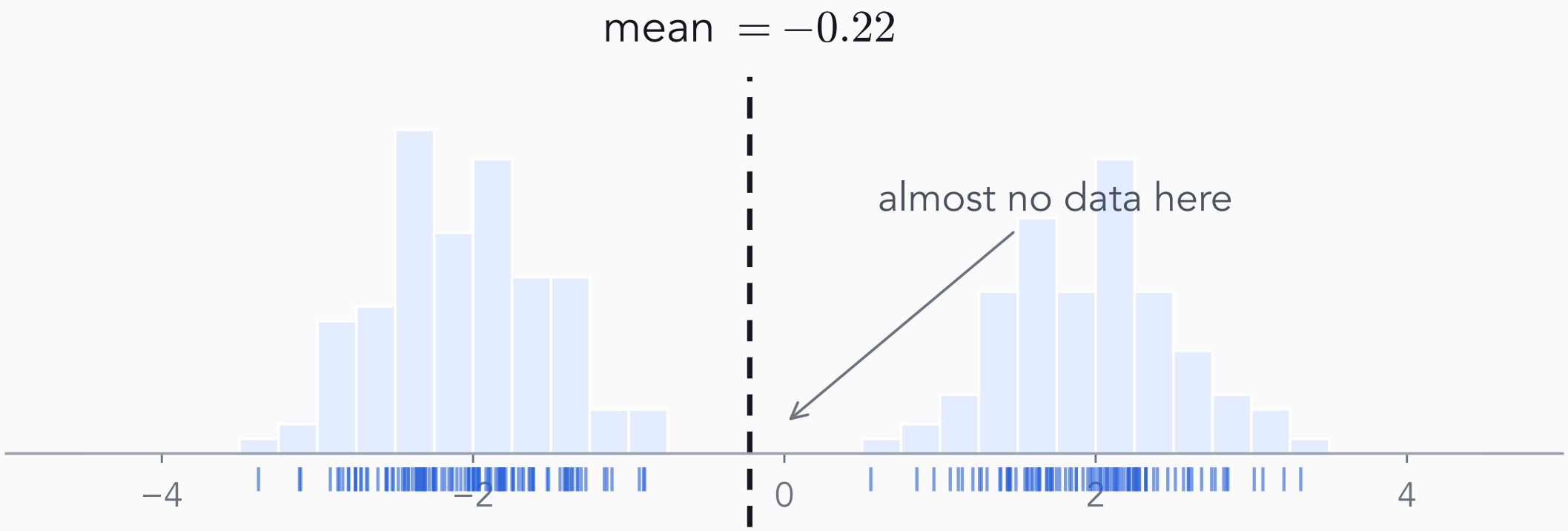
200 numbers from a recipe we keep hidden



> The model sees only these 200 numbers, not the recipe. Goal: learn from them, then produce a 201st that looks like it belongs.

A TEMPTING ANSWER

The average is a terrible sample



> **The mean is a number that almost never occurs. Generating needs the whole spread of the data, not one summary of it.**

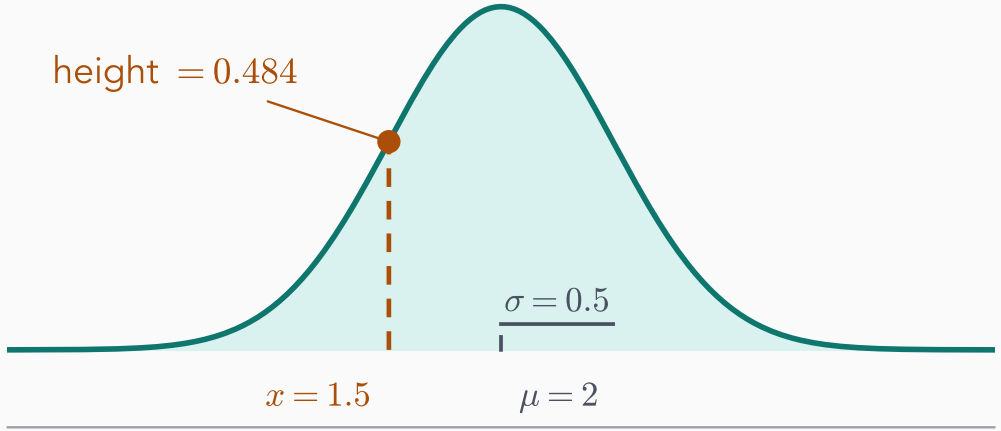
How probable is $x = 1.5$ under a bump at 2?

$$\mathcal{N}(x;\mu,\sigma^2) = \frac{1}{\sigma\sqrt{2\pi}}\exp\Big(-\frac{(x-\mu)^2}{2\sigma^2}\Big)$$

$$\frac{x-\mu}{\sigma} = \frac{1.5-2}{0.5} = -1$$

$$\exp\Big(-\tfrac{1}{2}\cdot(-1)^2\Big) = e^{-0.5} = 0.607$$

$$\frac{1}{0.5\sqrt{2\pi}} = 0.798 \qquad \Rightarrow \qquad 0.798 \times 0.607 = \mathbf{0.484}$$

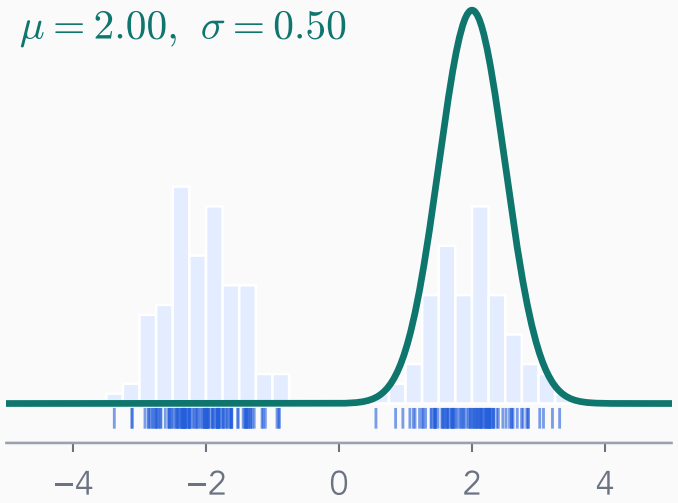


> The model's two parameters, μ and σ, decide how high the curve is at every x. Higher means more probable.

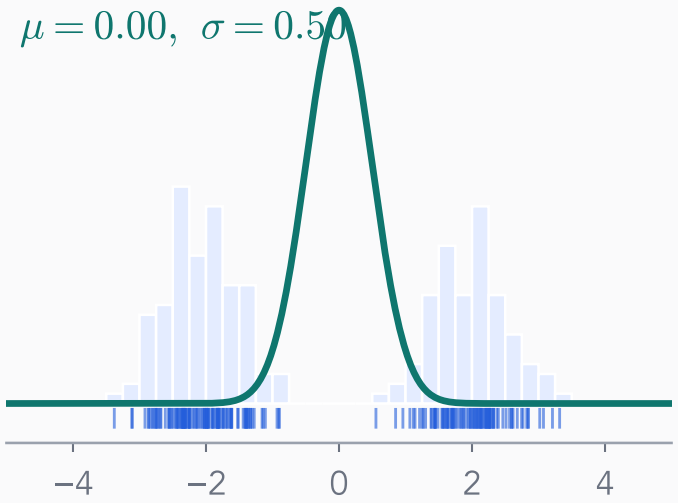
STEP 1 · LEARN

Score a choice of μ, σ on all 200 numbers

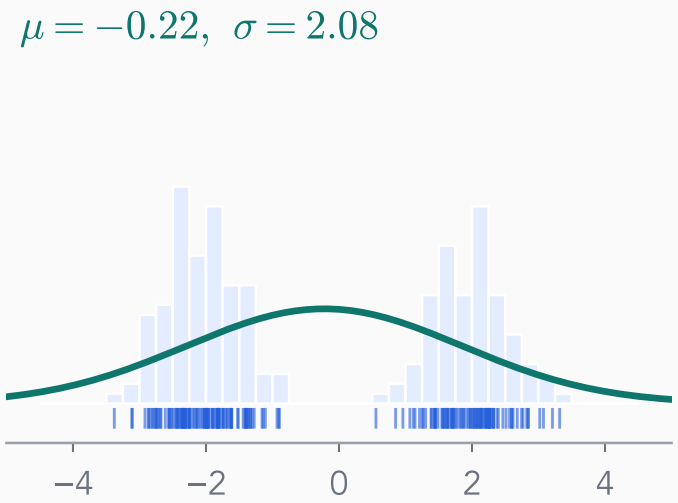
$$\log p_\theta(\text{data}) = \sum_{i=1}^{200} \log \mathcal{N}(x_i; \mu, \sigma^2), \qquad \theta = (\mu, \sigma)$$



$\log p = -3746.0$



$\log p = -1790.0$

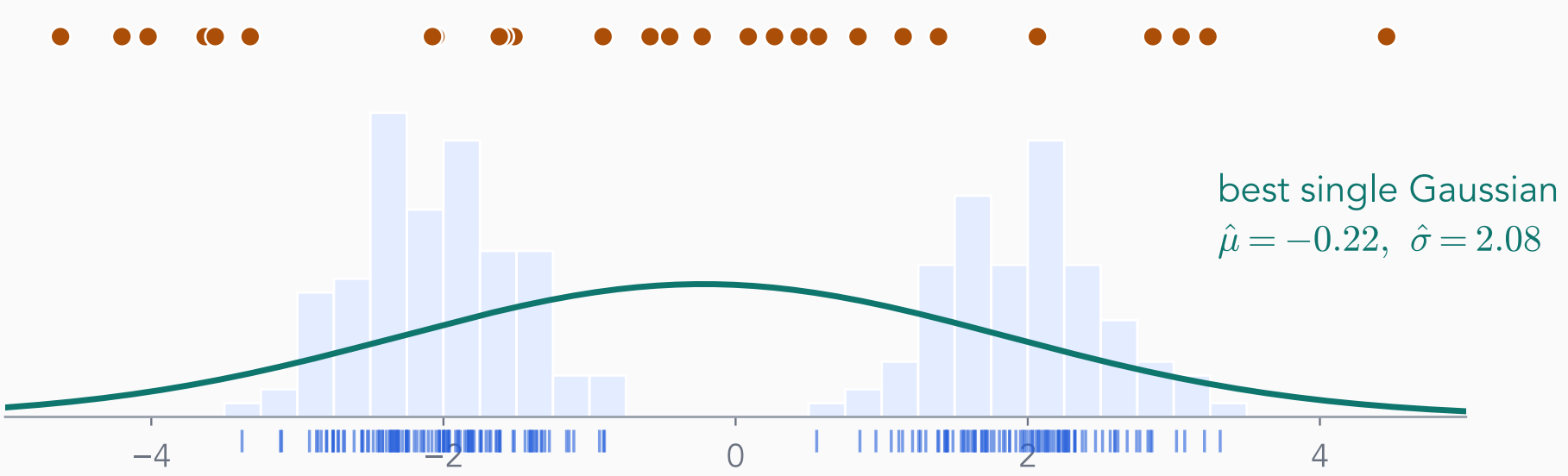


$\log p = -430.0$

> Learning = keep the θ with the highest score. That is **maximum likelihood**, the same rule that trained the name model.

STEP 1 · LEARN, THEN STEP 2 · GENERATE

The best Gaussian has a formula; then we sample from it



$$\hat{\mu} = \frac{1}{N}\sum_{i=1}^{N} x_i = -0.22 \qquad \hat{\sigma}^2 = \frac{1}{N}\sum_{i=1}^{N}(x_i - \hat{\mu})^2 = 4.31 \qquad x_{\text{new}} = \hat{\mu} + \hat{\sigma}\,\varepsilon, \quad \varepsilon \sim \mathcal{N}(0,1)$$

Step 2: 11 of 30 samples land in the empty gap. Learning worked; the model was too simple.

THE SAME FIT, IN PYTORCH

Learning = gradient descent on the negative log-likelihood

```
x = torch.tensor(data)                  # the 200 numbers
mu        = torch.tensor(1.0, requires_grad=True)
log_sigma = torch.tensor(0.0, requires_grad=True)
opt = torch.optim.Adam([mu, log_sigma], lr=0.05)

for step in range(300):
    dist = Normal(mu, log_sigma.exp())
    loss = -dist.log_prob(x).mean()
    opt.zero_grad(); loss.backward(); opt.step()

x_new = dist.sample((30,))              # step 2: generate
```

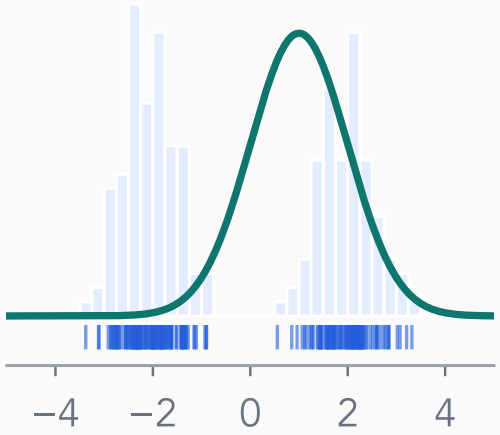
Result: $\mu = -0.22$, $\sigma = 2.08$, exactly the formula's answer. Then `sample` is step 2.

WATCH THE FIT HAPPEN

Four moments of that loop

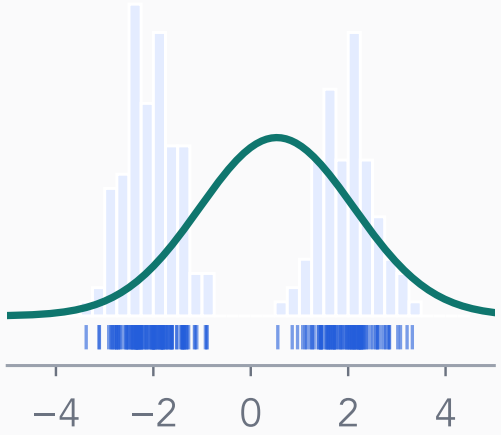
step 0: $\mu = 1.00,\ \sigma = 1.00$

loss $= 3.82$



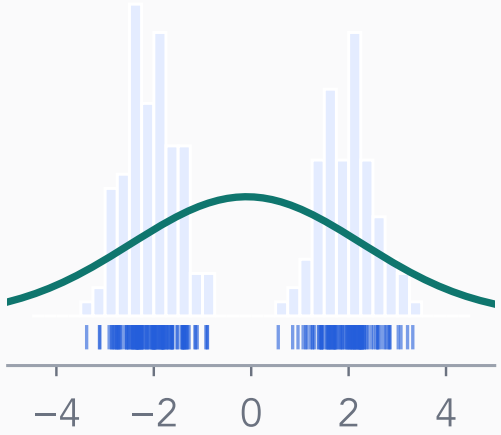
step 10: $\mu = 0.54,\ \sigma = 1.59$

loss $= 2.35$



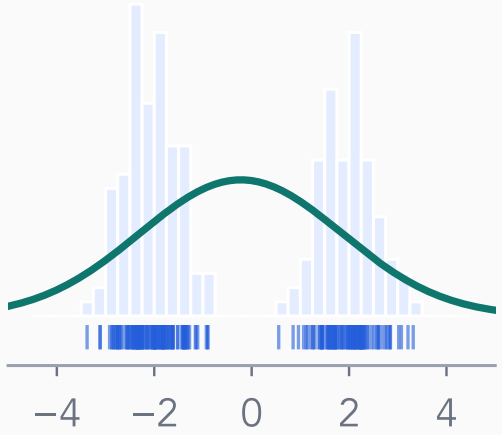
step 40: $\mu = -0.09,\ \sigma = 2.37$

loss $= 2.17$



step 300: $\mu = -0.22,\ \sigma = 2.08$

loss $= 2.15$



The curve starts in the wrong place, widens to cover both clumps, and stops at the best single bump. The loss falls from 3.82 to 2.15.

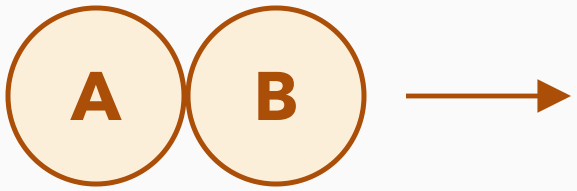
CHECKPOINT

So far: learn and generate work; one bump is not enough

	what we did	result
Learn	chose μ, σ to maximise $\log p_\theta(\text{data})$	$\hat{\mu} = -0.22,\ \hat{\sigma} = 2.08$
Generate	sampled $x = \hat{\mu} + \hat{\sigma}\varepsilon$	many samples in the gap
Fix	a richer model that can make two clumps	next slide

Hidden choice first, then the number

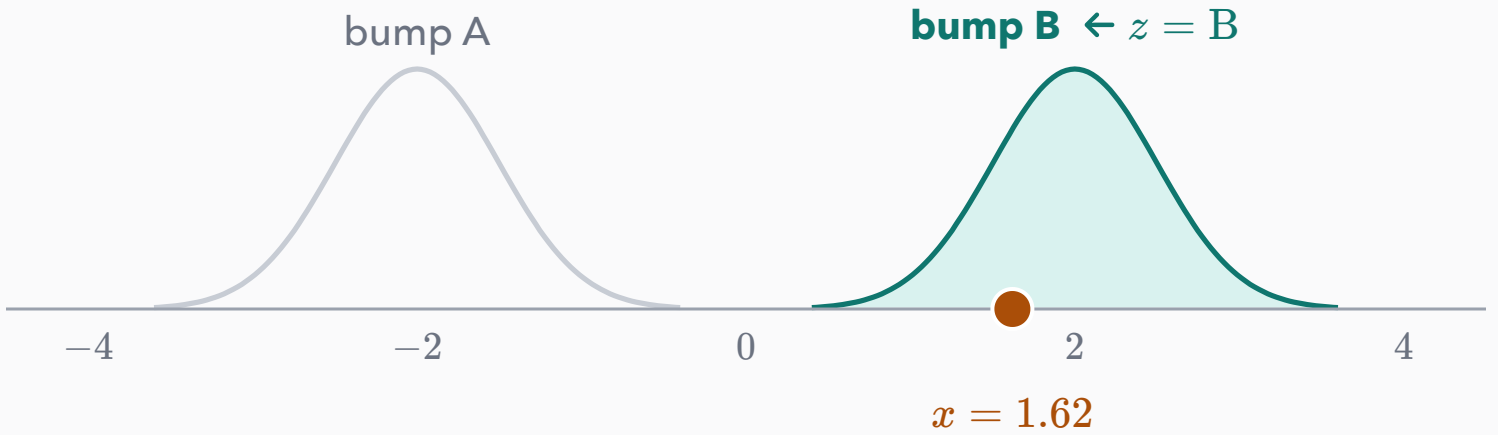
step 1 · hidden choice



flip a fair coin:

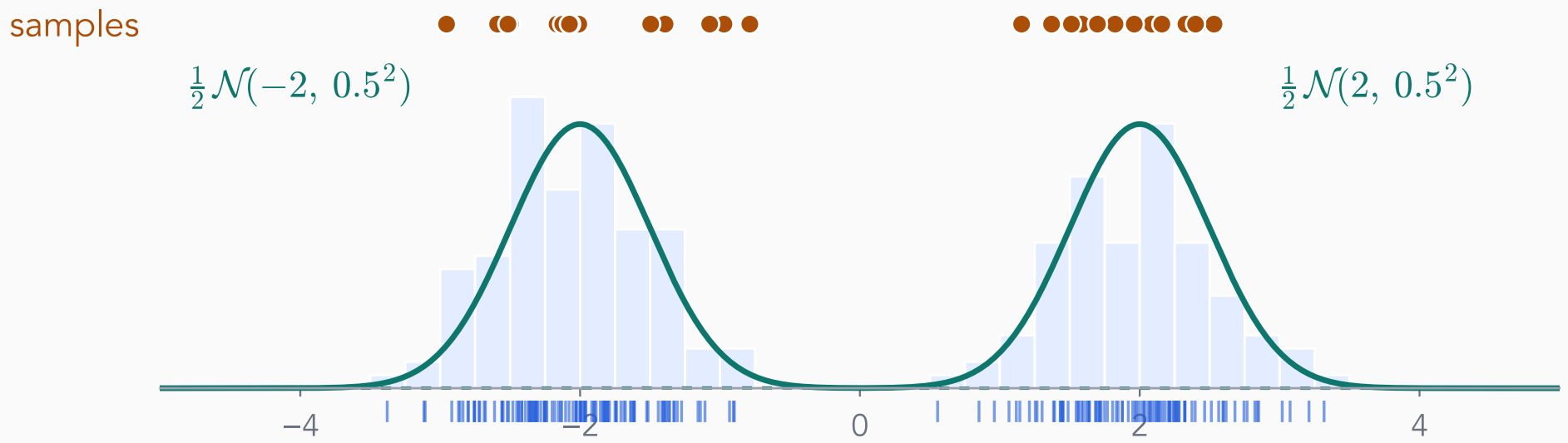
$p(z{=}\mathrm{A}) = p(z{=}\mathrm{B}) = \frac{1}{2}$

step 2 · draw x from the chosen bump



> This is how we will generate: run the two steps. The coin is never written down in the data, so it is a hidden (latent) variable z.

To score x, add up every way it could have been made



$$p(x) = \underbrace{\tfrac{1}{2}\,\mathcal{N}(x;\,-2, 0.5^2)}_{\text{made by bump A}} + \underbrace{\tfrac{1}{2}\,\mathcal{N}(x;\,2, 0.5^2)}_{\text{made by bump B}} = \sum_z p(z)\,p(x \mid z)$$

$$\text{at } x = 1.5: \quad \tfrac{1}{2}\cdot 0.000 + \tfrac{1}{2}\cdot 0.484 = 0.242$$

> One hidden coin flip made the model rich enough: the samples now fall in the bumps, not in the gap.

Same loop, a richer `log_prob`

```
logits = torch.zeros(K, requires_grad=True)   # bump weights
means  = torch.randn(K, requires_grad=True)
log_sd = torch.zeros(K, requires_grad=True)

def log_p(x):                                 # log p(x), shape [N]
    comp = Normal(means, log_sd.exp()).log_prob(x[:, None])   # [N, K]
    return torch.logsumexp(log_softmax(logits) + comp, dim=1)

loss = -log_p(x).mean()         # then the same 3-line update
```

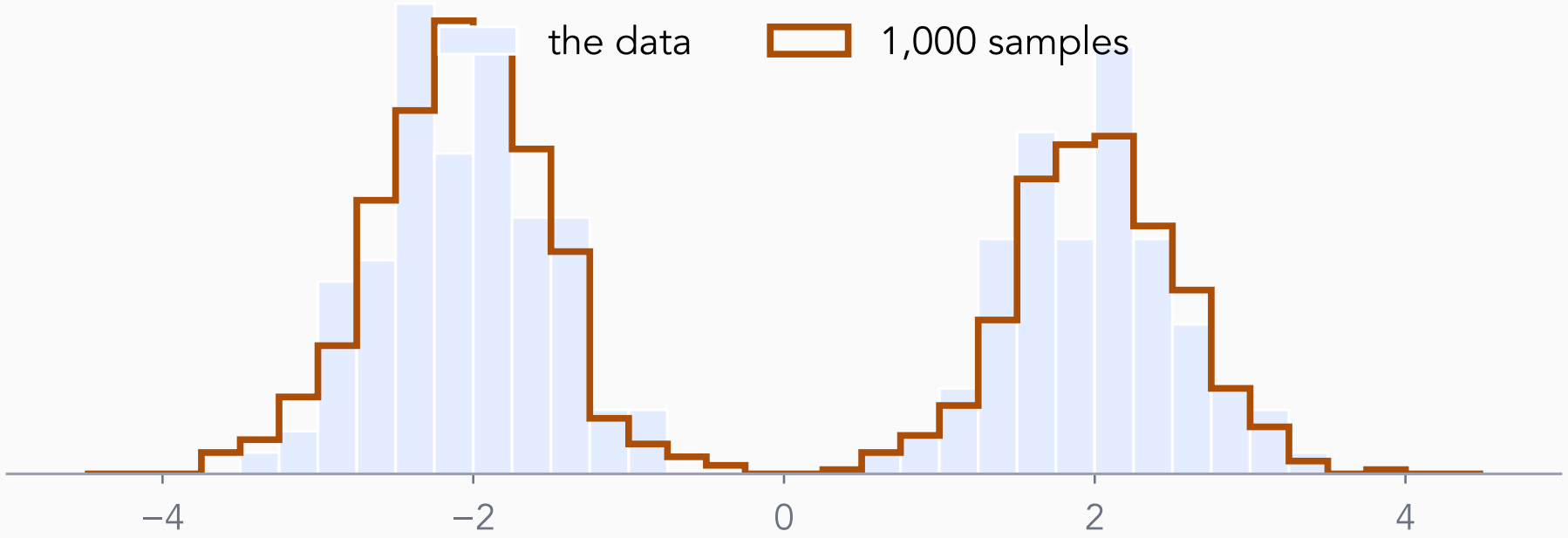
Training is unchanged. With $K = 2$ it finds centres -2.06 and 1.98, widths 0.52 and 0.54.

STEP 2 · GENERATE, FROM THE TWO-BUMP FIT

Sampling runs the story: choose a bump, then a number

```
weights = softmax(logits)
k = torch.multinomial(weights, 1000,
                      replacement=True)    # the coin
x_new = means[k] + log_sd.exp()[k] * torch.randn(1000)
```

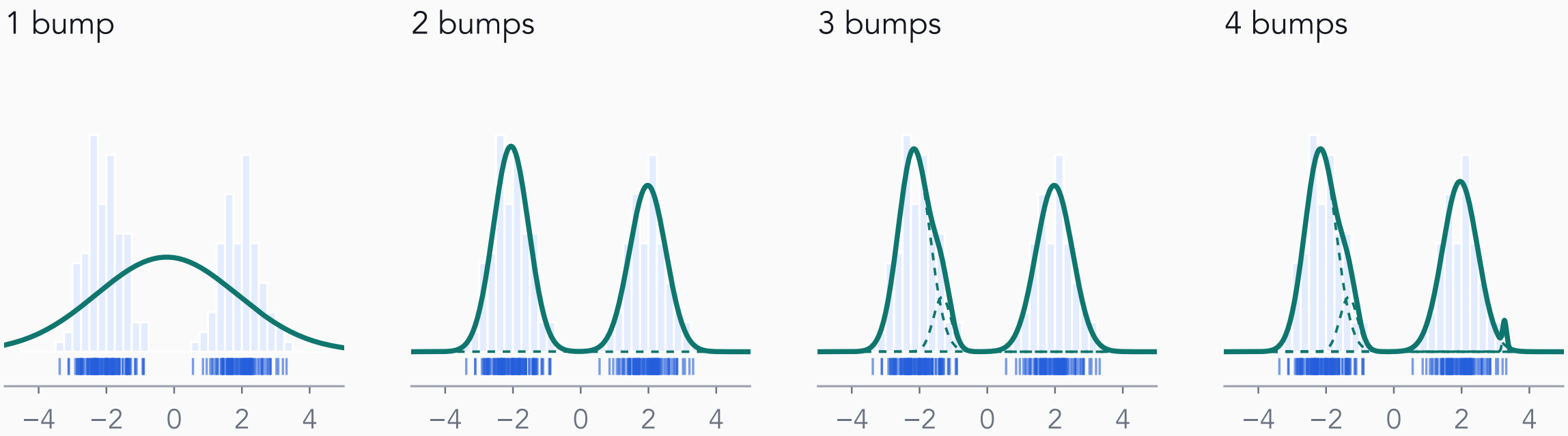
Step 2: draw a number from that bump: its centre plus its width times standard noise.



1,000 new numbers that follow the data, and none of them is a copy of a training number.

BUT HOW MANY BUMPS SHOULD WE FIT?

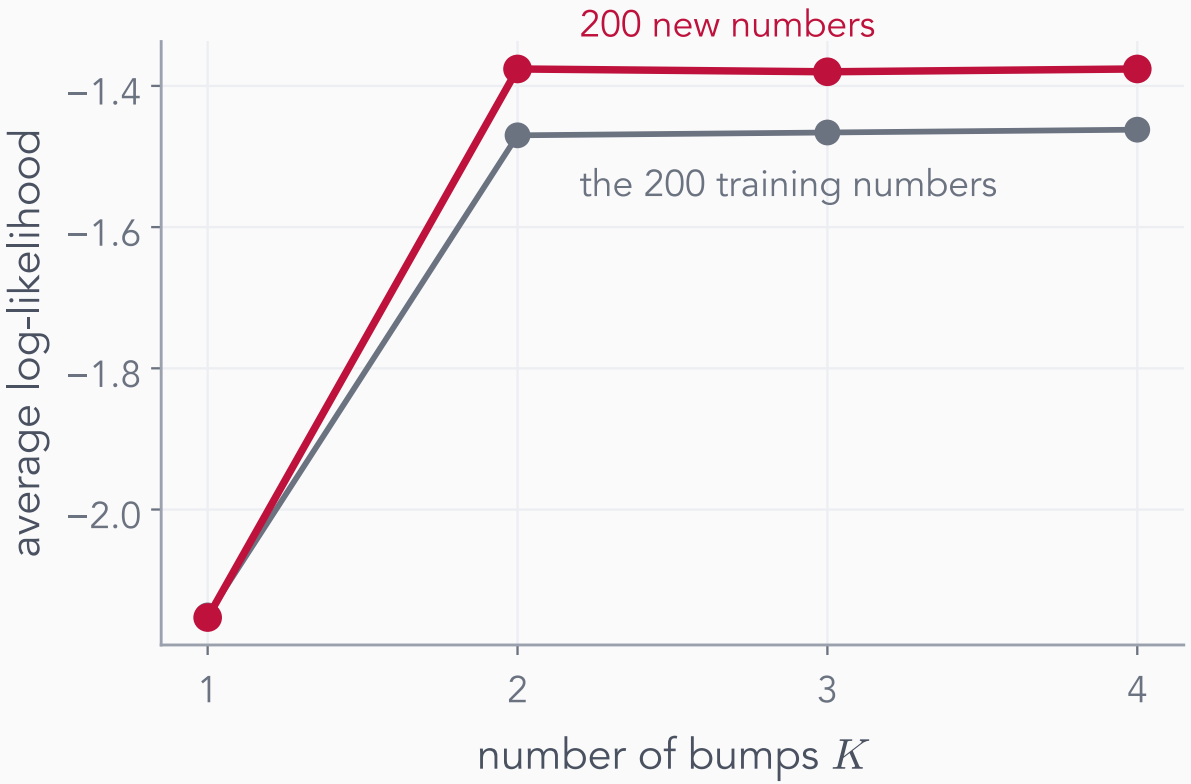
More bumps always fit the training data better



1, 2, 3, 4 bumps, each fitted with the same loop. The training score rises every time. So which one do we keep?

SCORE ON DATA THE FIT HAS NEVER SEEN

New data decides: two bumps



Draw **200 new numbers** from the same hidden recipe and score each fit on them.

1 → 2 bumps: $-2.153 \rightarrow -1.376$ per number. A real improvement.

3 or 4 bumps: the training score creeps up, the new-data score does not. The extra bumps fit noise in the 200 training numbers.

> **Choose the size of a model by how well it scores data it has not seen. That holds for bumps, and for networks.**

STEP 1 · LEARN, LOOKED AT CLOSELY

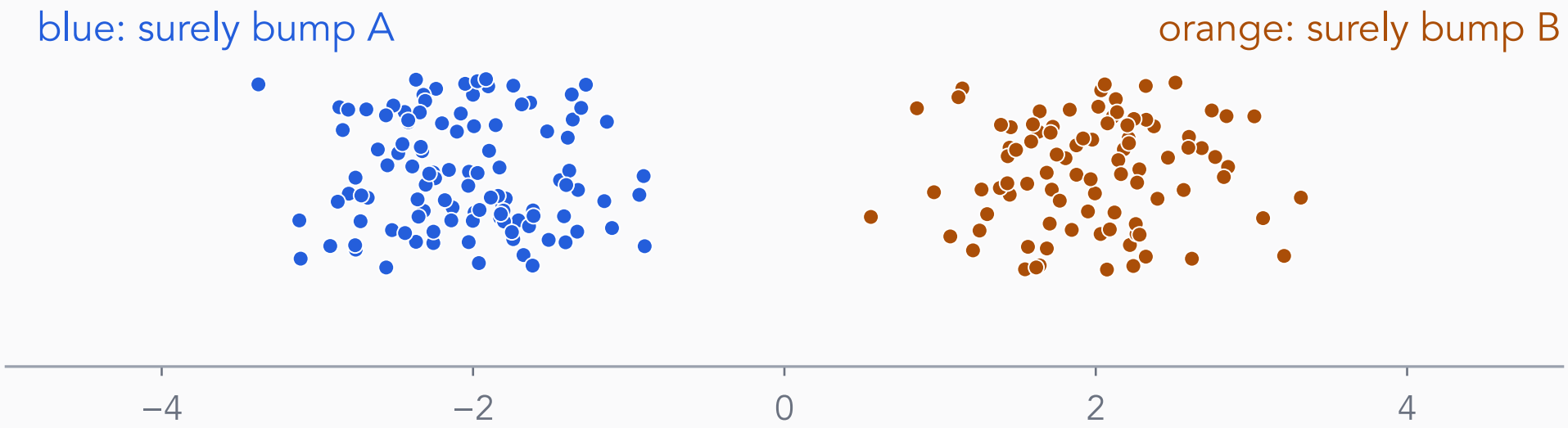
Hidden in the fit: which bump made each point?

If we knew the coin for every point

fitting is easy: average the A-points for bump A's centre, the B-points for bump B's.

We don't, so ask for each point

given this x, how probable is it that bump B made it? That is $p(\mathrm{B} \mid x)$.



Running the story backwards, from a number to its likely hidden cause, is called inference. The gradient descent we just ran was doing it all along: four slides on, we will see where.

THE BACKWARDS STEP, WORKED BY HAND

You see $x = 0.1$. Which bump made it?

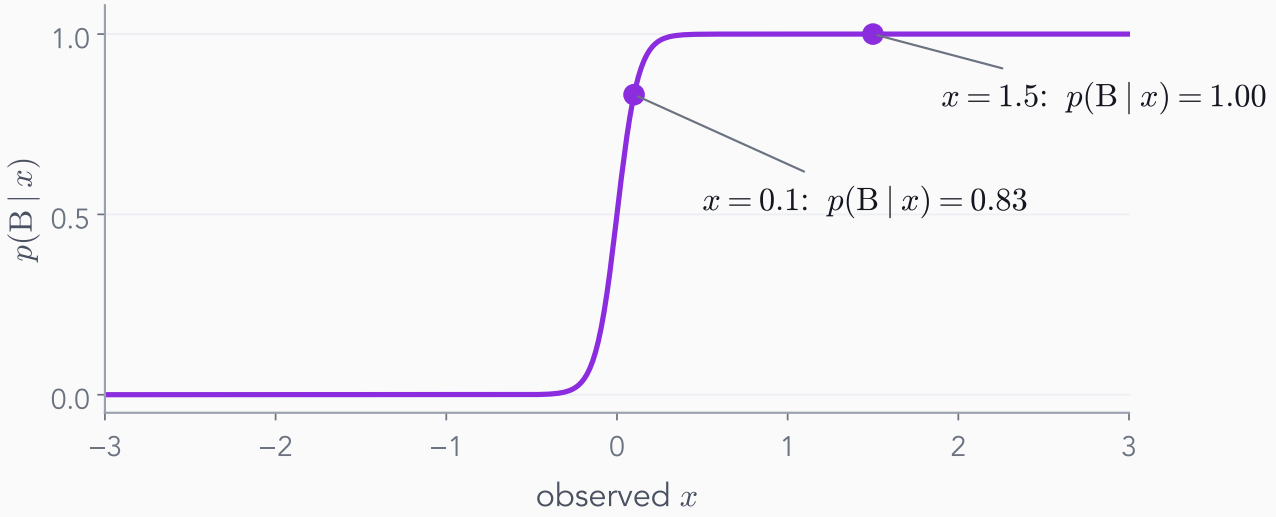
bump z	prior $p(z)$	likelihood $p(x \mid z)$	prior × likelihood	÷ total = posterior
A, centre -2	0.5	1.18×10^{-4}	5.89×10^{-5}	0.168
B, centre 2	0.5	5.84×10^{-4}	2.92×10^{-4}	**0.832**
total $= p(x)$			3.51×10^{-4}	**1.000**

$$p(\mathrm{B} \mid x) = \frac{p(\mathrm{B})\, p(x \mid \mathrm{B})}{p(x)} = 0.832$$

> **Bayes' rule is just this table: multiply, then divide by the total. Note the total is exactly the $p(x)$ of the two-bump formula.**

DO IT FOR EVERY x

The backwards step, as a curve



Repeat the table for every x and plot $p(\mathrm{B} \mid x)$.

Far right: certainly B. Far left: certainly A. Near 0: unsure.

For these two bumps the curve is exactly $\mathrm{sigmoid}(16x)$: a logistic regression in disguise.

These are the colours on the 200 points two slides ago.

THE BACKWARDS STEP, IN ONE LINE

Posterior = softmax of (log prior + log likelihood)

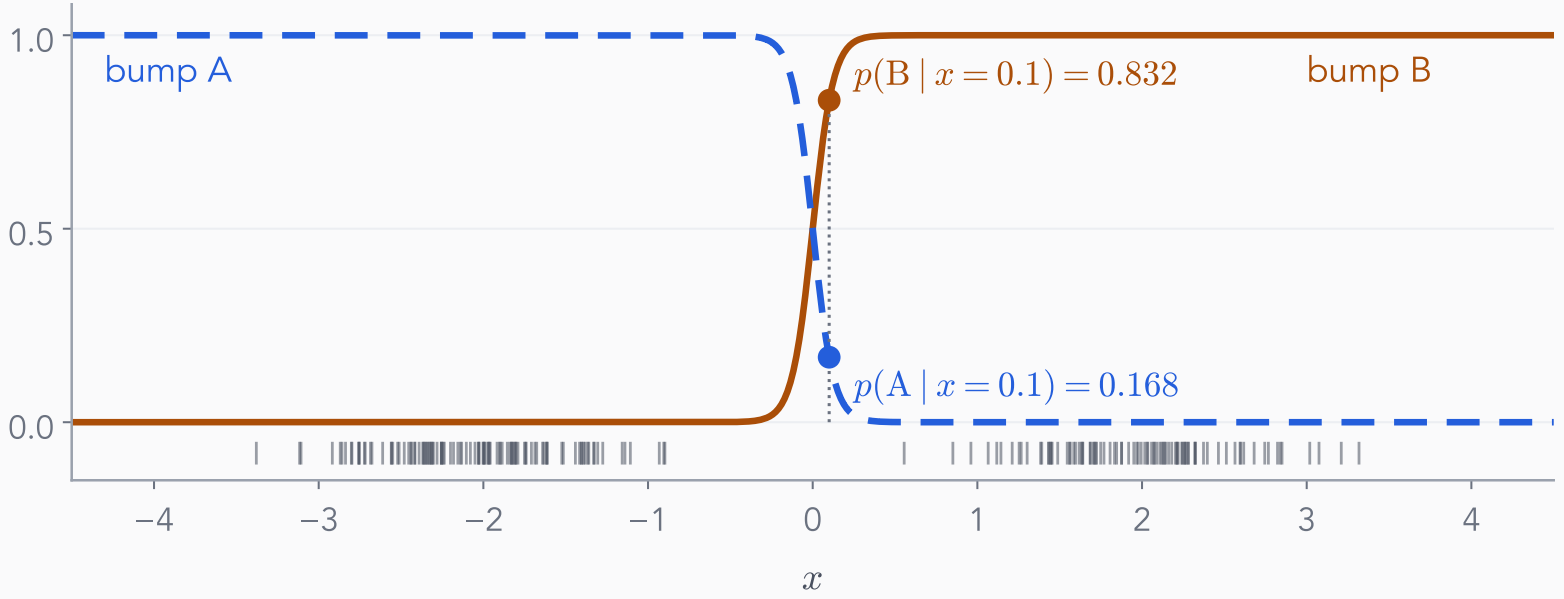
```
log_joint = log_softmax(logits) + Normal(means, log_sd.exp()).log_prob(x)   # [K]
posterior = softmax(log_joint)                    # divide by the total: Bayes
```

With the fitted bumps, at $x = 0.1$: $p(\mathrm{A} \mid x) = 0.094$, $p(\mathrm{B} \mid x) = 0.906$. (With the true recipe it was 0.832: the fitted bumps are slightly different.)

Softmax is "exponentiate, then divide by the total": exactly the last column of the Bayes table.

The gradient of $\log p(x)$ is weighted by the posterior

$$\frac{\partial \log p(x)}{\partial \mu_k} = p(k \mid x) \cdot \frac{x - \mu_k}{\sigma_k^2}$$



At $x = 0.1$: bump B's centre is pulled by $0.832 \times (0.1 - 2)/0.25 = -6.32$, bump A's by $0.168 \times (0.1 + 2)/0.25 = 1.41$.

> **Each point pulls each bump in proportion to how likely that bump made it. Learning needs the posterior for every point: a 2-row table here, an intractable patch for images.**

FOUR WORDS FOR THE REST OF THE LECTURE

Prior, likelihood, evidence, posterior

name	symbol	question it answers	at $x = 0.1$
prior	$p(z)$	what do we believe about z before seeing x?	0.5, 0.5
likelihood	$p(x \mid z)$	if z were known, how probable is this x?	5.84×10^{-4} for B
evidence	$p(x)$	how probable is x overall, summing over z?	3.51×10^{-4}
posterior	$p(z \mid x)$	having seen x, which z explains it?	B: 0.832

Generating uses the prior and the likelihood. Learning needs the evidence and the posterior.

END OF PART 1

What we did, and what comes next

WHAT WE DID

1. **Learn** = choose parameters that make the data probable.
2. **Generate** = sample from the learned model.
3. One bump was too simple; a hidden coin z fixed it.
4. Scoring x sums over z; learning needs the backwards step $p(z \mid x)$.
5. In code: one loop, `loss = -log_p(x).mean()`; choose the size on new data.

NEXT: IMAGES

784 numbers per example instead of one. Before any probability, a simpler idea: squeeze each image into a few numbers and rebuild it. That is the autoencoder.

Part 3 returns to this probabilistic story, with a code z in place of the coin.

PART 2 OF 7

First attempt: an autoencoder

Part 1 modelled single numbers; an image has 784. Before any probability, a simpler idea: squeeze each image into a few numbers with an **encoder**, rebuild it with a **decoder**, and see how far that gets us towards generating.

✓ Distributions and fitting

2 **Autoencoders**

3 The probabilistic view

4 KL divergence

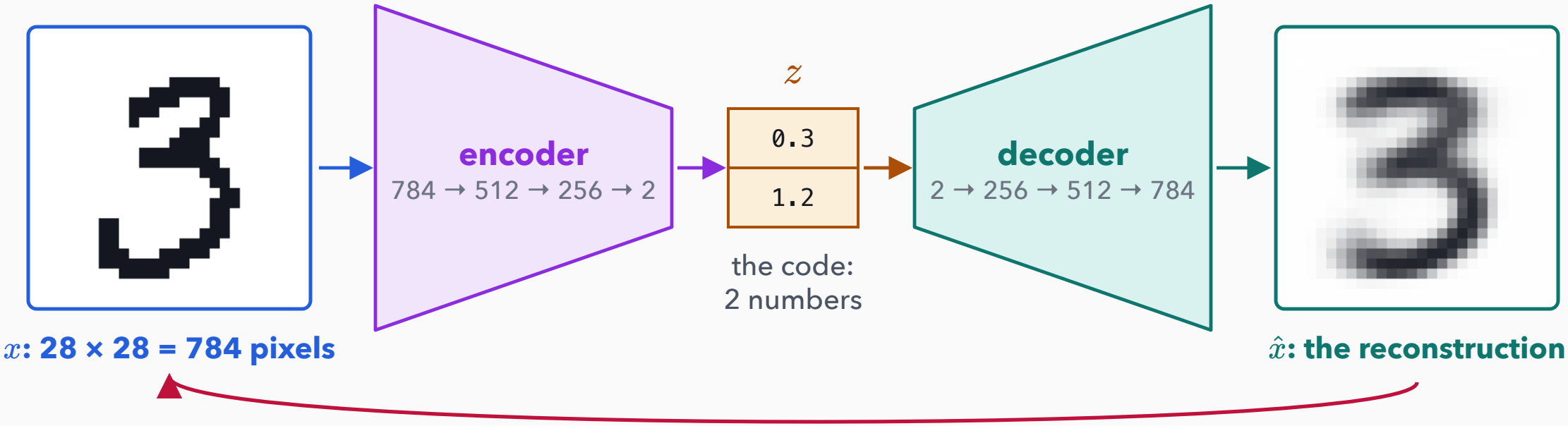
5 Variational inference and the ELBO

6 Reparameterisation

7 The VAE on MNIST

SQUEEZE, THEN REBUILD

An autoencoder on our 3



> No labels needed: the image is its own target. The 2 numbers in the middle are all the decoder gets.

COMPARING THE REBUILD WITH THE ORIGINAL

Binary cross-entropy, one pixel at a time

$$\text{loss}(x, \hat{x}) = -\sum_{i=1}^{784} \Big[x_i \log \hat{x}_i + (1 - x_i) \log(1 - \hat{x}_i) \Big]$$

pixel	true value	predicted	loss
ink, predicted well	1	0.90	$-\log 0.90 = 0.11$
paper, predicted well	0	0.10	$-\log 0.90 = 0.11$
ink, missed	1	0.20	$-\log 0.20 = 1.61$

Pixels are 0 or 1 here (we binarise MNIST). Later we will see this is exactly a log-likelihood.

Two small networks and one loss

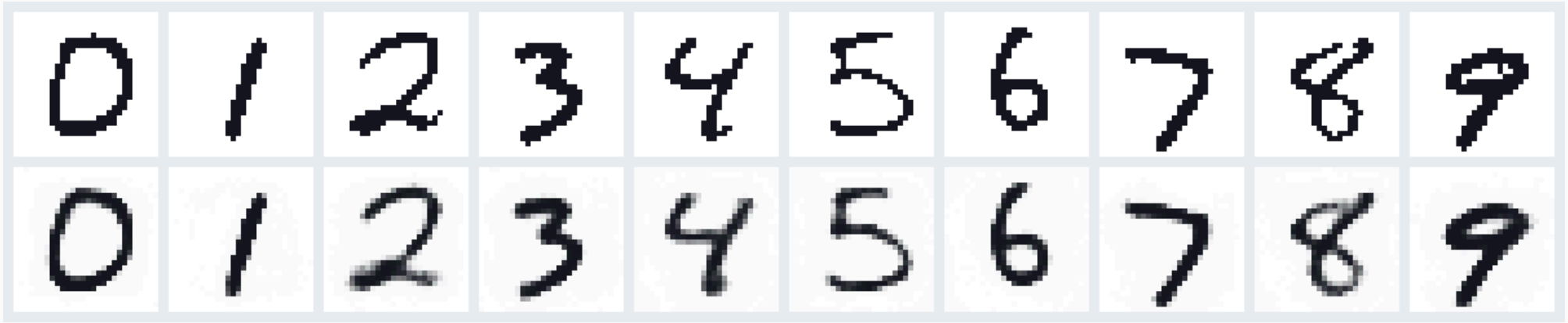
```
encoder = nn.Sequential(nn.Flatten(),
    nn.Linear(784, 512), nn.ReLU(),
    nn.Linear(512, 256), nn.ReLU(), nn.Linear(256, d))
decoder = nn.Sequential(nn.Linear(d, 256), nn.ReLU(),
    nn.Linear(256, 512), nn.ReLU(), nn.Linear(512, 784))

z      = encoder(x)                    # [B, d]   the code
logits = decoder(z)                    # [B, 784] one per pixel
loss = F.binary_cross_entropy_with_logits(
          logits, x.flatten(1), reduction="none").sum(1).mean()
```

Loss: binary cross-entropy summed over the 784 pixels of each image, then averaged over the batch. Train with the same three-line update as the bumps.

TRAINED ON 60,000 DIGITS

It reconstructs very well



top: test digits it never saw

bottom: decoded from a 16-number code

784 pixels squeezed into 16 numbers and back. Average loss 44.9 nats per image. Can we now make new digits?

HOW NARROW CAN THE MIDDLE BE?

2 numbers or 16 numbers per image

a 2-number code · loss 128.1 nats per image



a 16-number code · loss 44.9 nats per image



Each pair of rows: test digits on top, reconstructions below. Two numbers keep the digit's identity; sixteen also keep its slant and stroke.

ONE IDEA, SEVERAL VARIATIONS

The autoencoder family

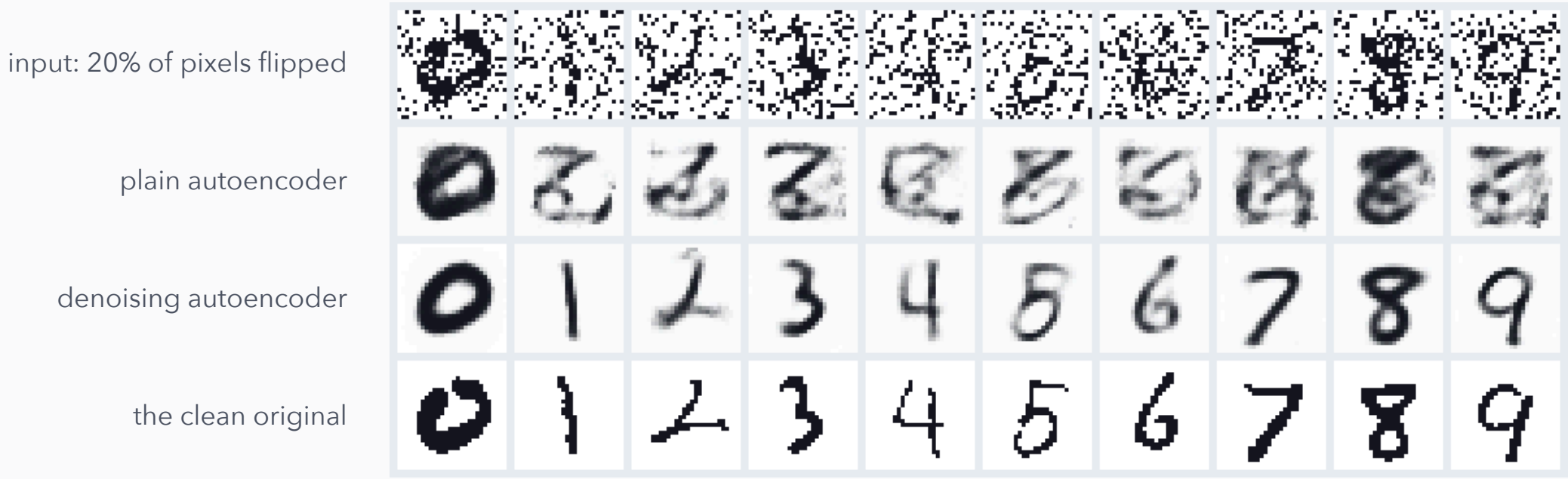
variant	what changes	what it encourages
plain	nothing: compress, rebuild	a short code that keeps what matters
denoising	corrupt the input, rebuild the **clean** image	features that ignore noise
sparse	penalty: only a few code units may be on	each unit means one part of a digit
contractive	penalty: the code barely moves when the input moves a little	smooth, robust codes
variational	each image becomes a **cloud** of codes near $\mathcal{N}(0, I)$	a code space you can **sample** from

> **The first four change the loss or the input. The last one changes what a code is, and it is where today is going.**

Following Lilian Weng, "From Autoencoder to Beta-VAE" (2018)

VARIANT 1 · DENOISING AUTOENCODER

Feed it a damaged image, ask for the clean one



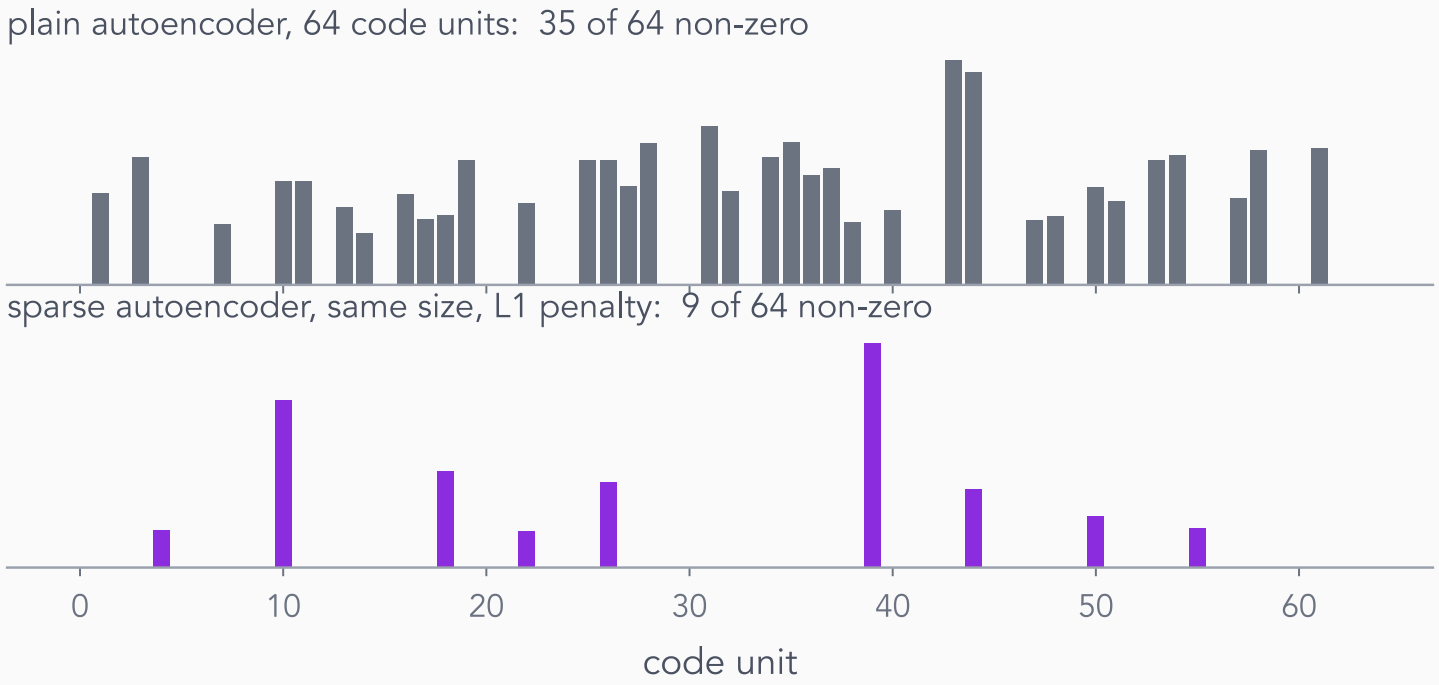

```
noisy = flip_pixels(x, rate=0.2)                    # the only new line
loss  = bce_with_logits(decoder(encoder(noisy)), x)  # target: the CLEAN x
```

The plain autoencoder copies the noise. Trained to undo damage, the code must hold the digit, not the specks.

VARIANT 2 · SPARSE AUTOENCODER

A wide code, but only a few units allowed on

$$\text{loss} = \text{reconstruction} + \lambda \sum_{j=1}^{64} |h_j|, \qquad h = \text{ReLU}(\text{encoder}(x))$$



The 64-number code of our 3. Averaged over test digits: 34.9 units on without the penalty, 8.8 with it.

VARIANT 3 · CONTRACTIVE AUTOENCODER (NOT TRAINED HERE)

Nudge the input; the code should barely move

$$\text{loss} = \text{reconstruction} + \lambda \left\| \frac{\partial h}{\partial x} \right\|_F^2 = \text{reconstruction} + \lambda \sum_{j,i} \left(\frac{\partial h_j}{\partial x_i} \right)^2$$

Read the penalty

for every code unit h_j and every pixel x_i: how much does h_j change if x_i changes a little? Add up the squares.

What it buys

two slightly different 3s get almost the same code. Nearby inputs, nearby codes: a first step towards a smooth code space.

WHAT AUTOENCODERS ARE USED FOR

Compress, clean, search, and spot the odd one out

Compress: 784 pixels → 16 numbers, 49× fewer.

Clean: the denoising slide.

Search: similar images have nearby codes, so search the codes.

Spot anomalies: an image the model has never seen anything like gets a large reconstruction error.



top: typical test digits, loss ≈ 40.5

bottom: the 8 worst of 10,000, loss up to 218.0

> **Useful, all of it. But none of it generates a new image. Can the decoder do that on its own?**

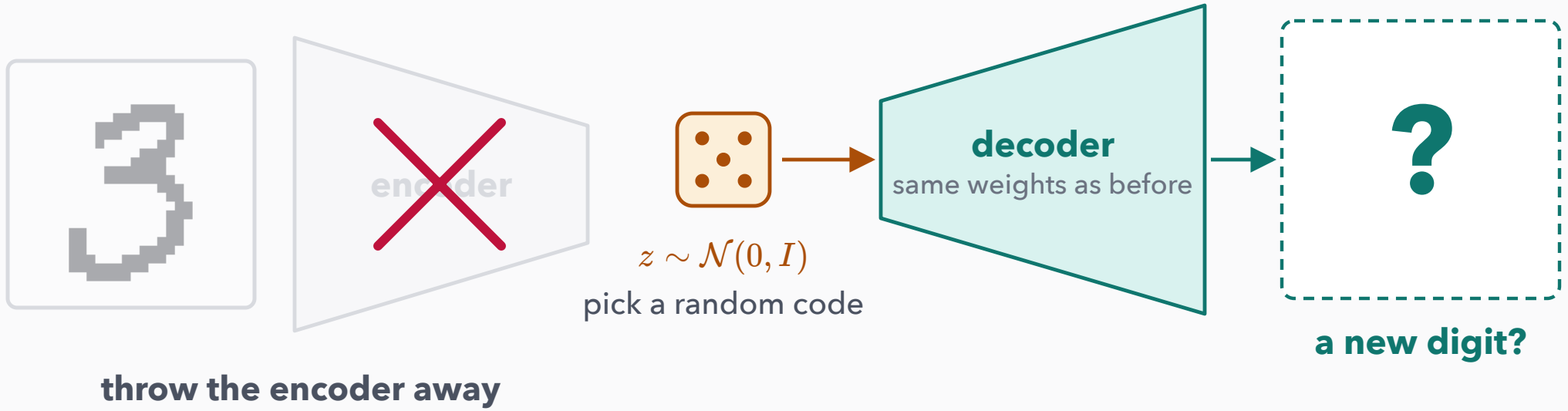
PART 2, SECOND HALF

Can an autoencoder generate?

The decoder turns codes into digits. If we pick a code ourselves, do we get a new digit?

THE OBVIOUS IDEA

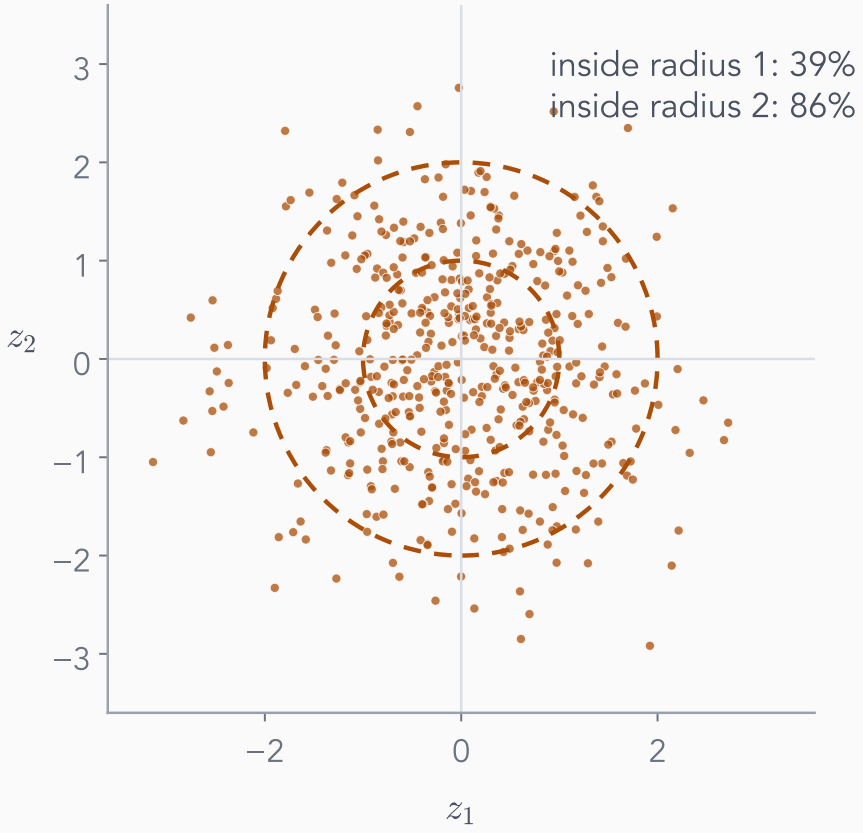
Keep the decoder, feed it a random code



The decoder turns codes into digits. So a random code should give a random digit. Right?

WHAT IS A "RANDOM CODE"?

$z \sim \mathcal{N}(0, I)$: two independent bell curves



Draw z_1 from a standard bell curve $\mathcal{N}(0, 1)$: mean 0, spread 1. Draw z_2 the same way, independently.

Together that is written $\mathcal{N}(0, I)$: the **0** says the centre is $(0, 0)$; the **I** (identity matrix) says each number has spread 1 and the two do not depend on each other.

500 draws, plotted: most land within 2 of the origin, almost none beyond 3.

> **Why this choice? It is the simplest distribution we can sample from. The decoder, though, was never told about it.**

WHAT ACTUALLY COMES OUT

Random codes give scribbles

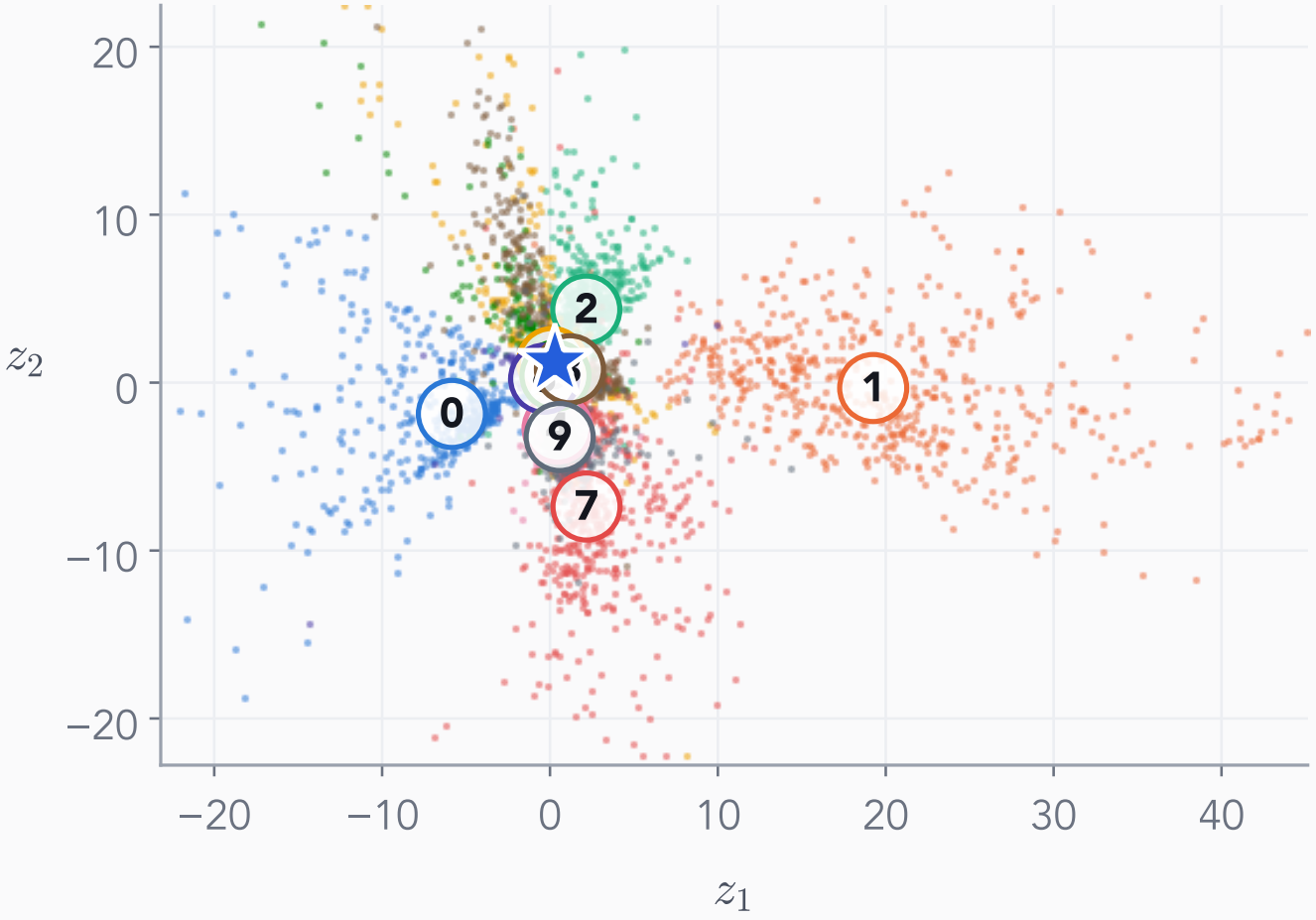


ten codes drawn from $\mathcal{N}(0, I)$ in 16 dimensions, through the trained decoder

The decoder only ever saw codes made by the encoder. Where do those codes live?

LOOK INSIDE (2-NUMBER VERSION)

Where 5,000 test digits land



Each dot is one test digit's code, coloured by its true label. The star is our 3.

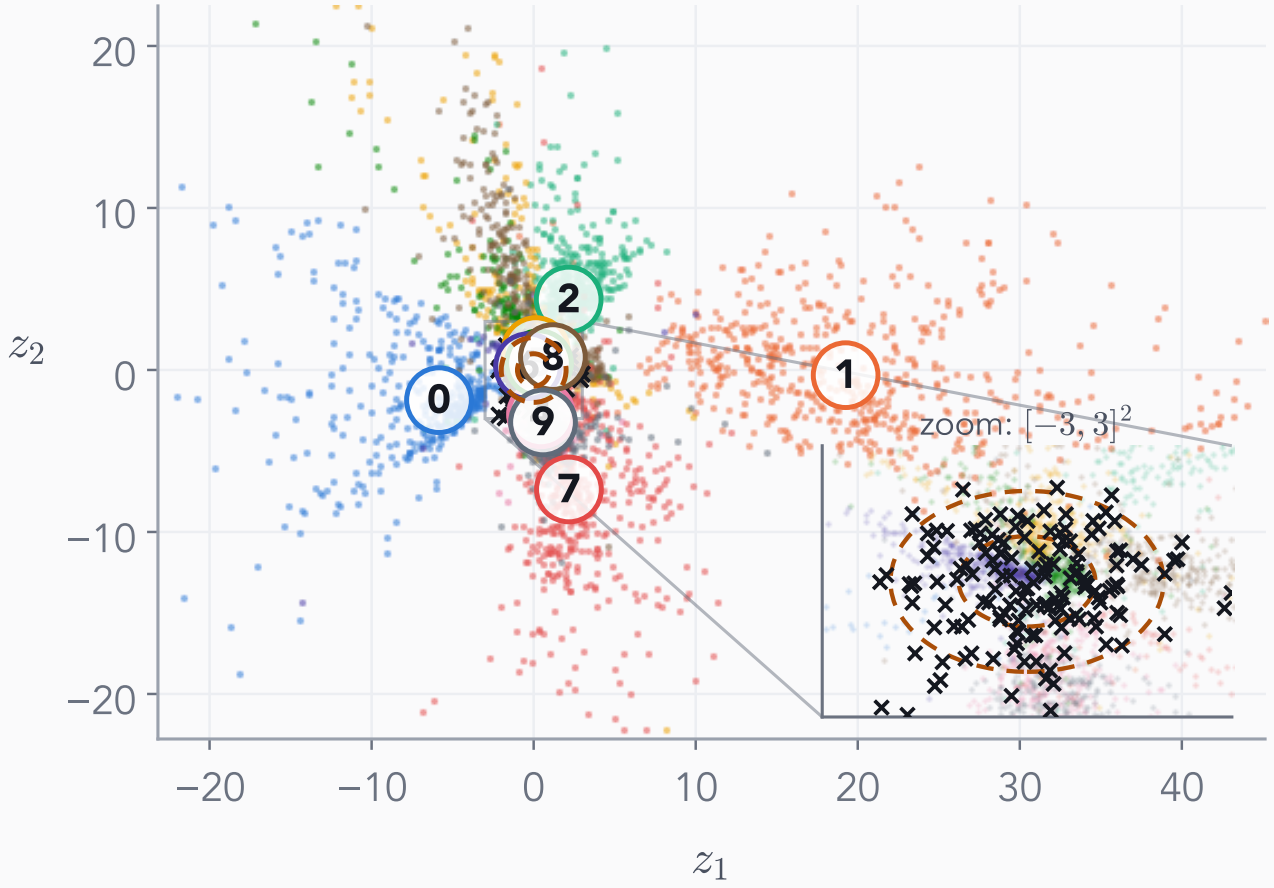
Same digits cluster together. Nobody asked for that; it just helps reconstruction.

Look at the axes: z_1 runs from −18.5 to 40.4, z_2 from −19.7 to 19.4. Nothing made the codes stay near the origin.

The code space has an arbitrary shape, scale and position.

OVERLAY RANDOM CODES

Random draws all land in one small spot



Now add 150 random codes from $\mathcal{N}(0, I)$ (black ×), the kind we fed the decoder two slides ago. Dashed circles: radius 1 and 2. On this map they all fall into one small crowded patch (zoomed, bottom right).



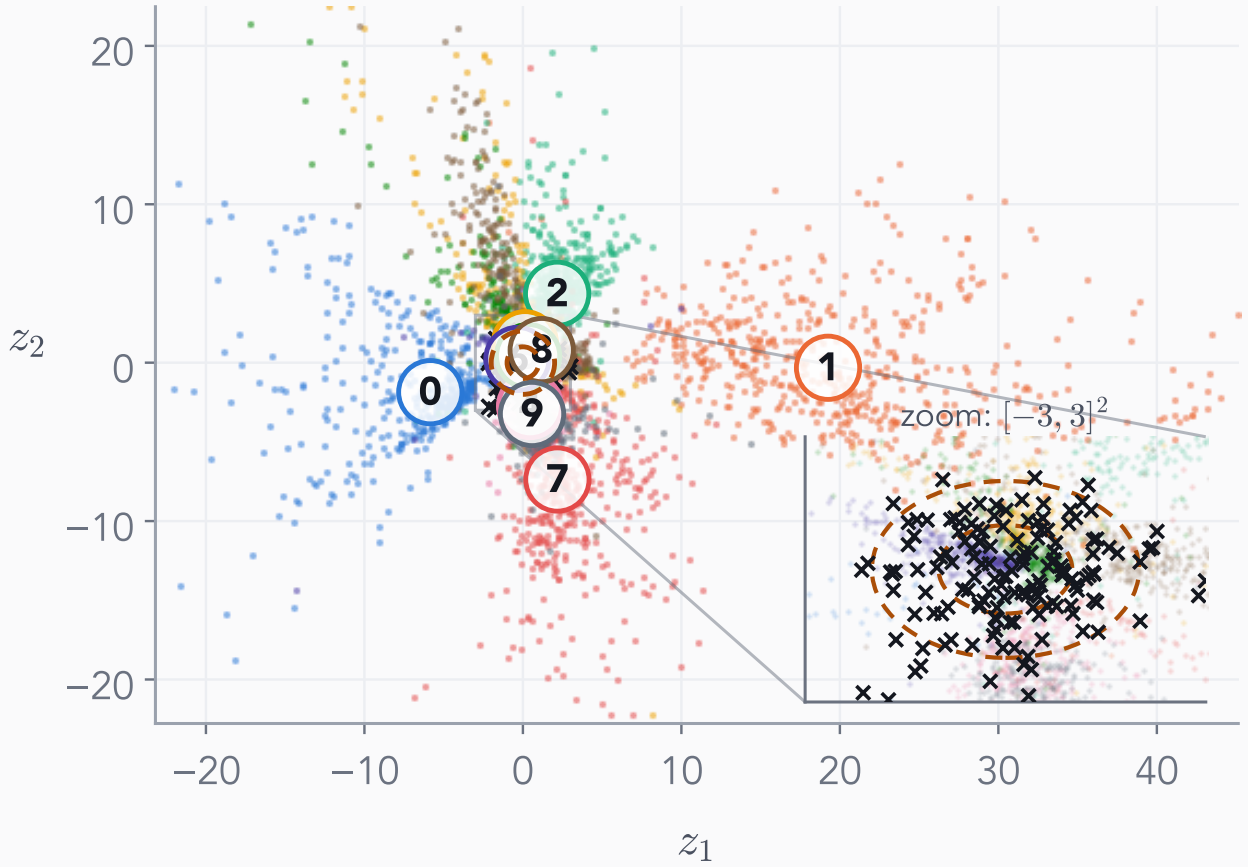
ten of them decoded: some digits, some smudges

And we can never generate a 1: the 1s live near $z_1 \approx 20$.

To sample, we need to know where the codes are, and every point there must decode to something sensible.

TRY IT: THE TRAINED AUTOENCODER, IN YOUR BROWSER

Pick a code, any code, and decode it

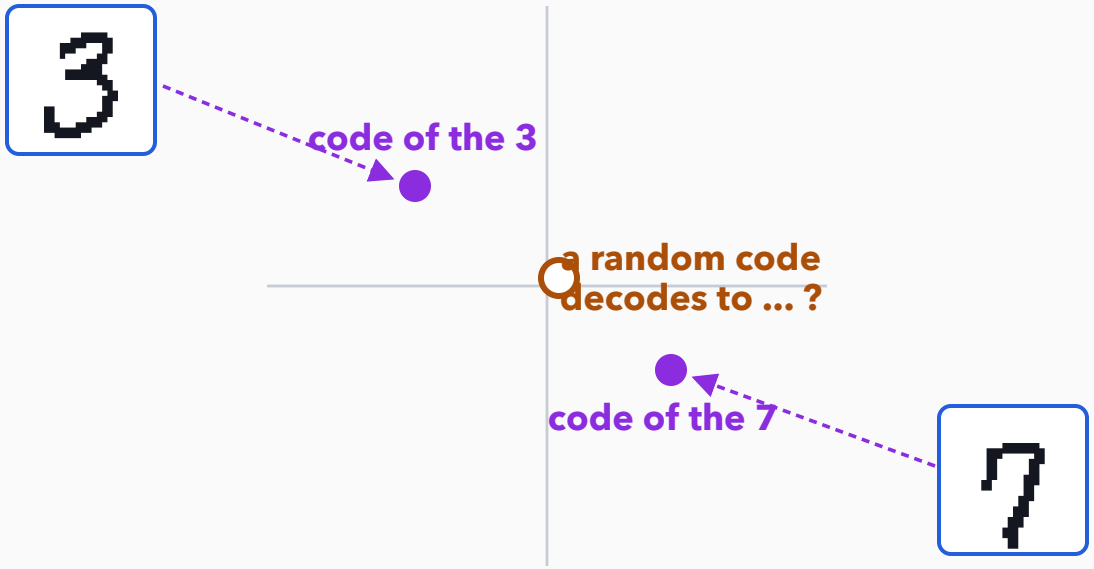


A real digit's code decodes well. A random code usually gives a smudge: the decoder only learned the dotted regions.

WHAT WE WANT INSTEAD

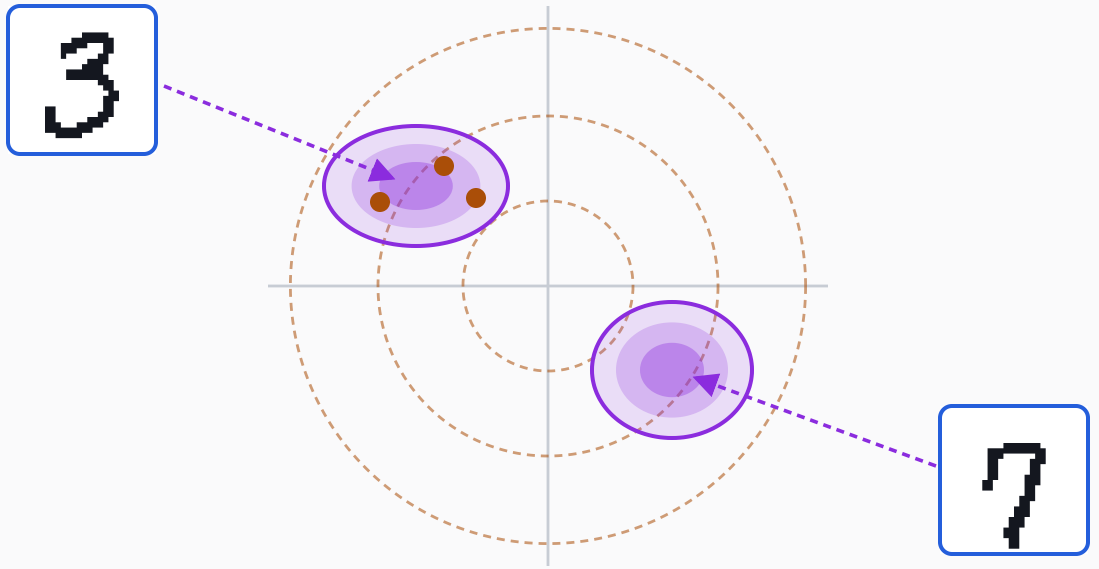
Encode to a cloud, not a point

autoencoder: each image → one point



training only ever used the two purple points

VAE: each image → a small cloud



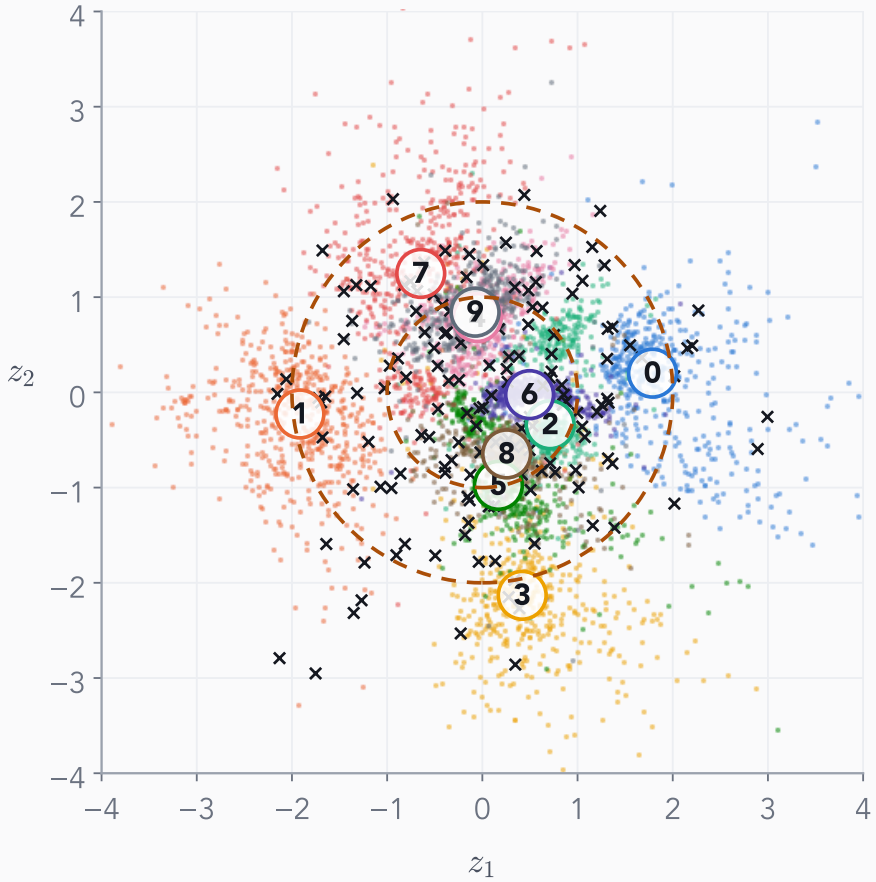
training: every code in the 3's cloud must decode back to the 3

Known: pull every cloud towards $\mathcal{N}(0, I)$, so random draws land on real codes.

Smooth: every point in a cloud must decode to that image, so neighbourhoods are meaningful.

A PREVIEW OF WHERE WE ARE GOING

The same plot for a VAE



Same data, same 2-number code, same random draws. Now the codes sit inside $\mathcal{N}(0, I)$.



the same kind of random codes, decoded

> **To get here: make the model probabilistic (Part 3), measure how close two distributions are (Part 4), and derive the loss (Part 5).**

END OF PART 2

What we did, and what comes next

WHAT WE DID

1. An encoder squeezes an image into a short code; a decoder rebuilds it. The image is its own target.
2. Random codes from $\mathcal{N}(0, I)$ decode to scribbles.
3. The codes spread over an arbitrary range, and the decoder never saw the space between them.
4. Variants change the input (denoising) or add a penalty (sparse, contractive).
5. The wish: encode each image to a **cloud**, and keep every cloud near $\mathcal{N}(0, I)$.

NEXT: THE PROBABILISTIC VIEW

Turn the decoder into a probability model with a prior over codes, and ask what it would take to train it by maximum likelihood, as in Part 1.

PART 3 OF 7

The probabilistic view: a decoder is a likelihood

The autoencoder could not generate because nothing said where codes live. Part 1 had exactly what is missing: a **prior** over the hidden choice. Put Part 1's story and Part 2's decoder together.

✓ Distributions and fitting

✓ Autoencoders

3 **The probabilistic view**

4 KL divergence

5 Variational inference and the ELBO

6 Reparameterisation

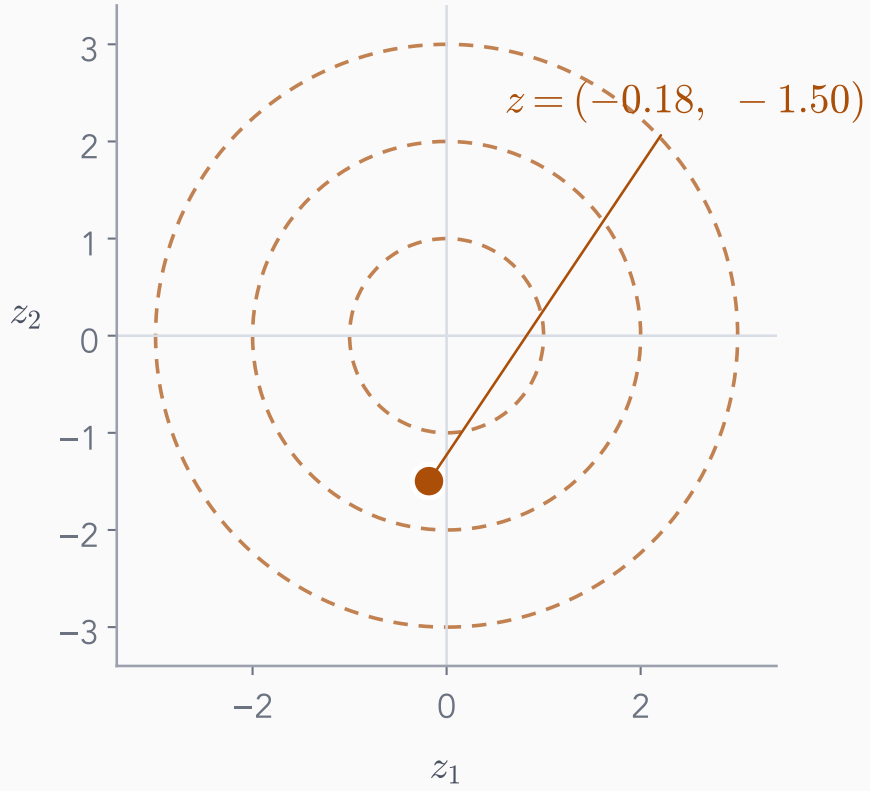
7 The VAE on MNIST

Same story, bigger pieces

	two bumps (so far)	images (from now on)
hidden z	a coin: A or B	2 real numbers (z_1, z_2), drawn from $\mathcal{N}(0, I)$
$p(x \mid z)$	a bump at -2 or $+2$	a neural network, the **decoder**, with weights θ
observed x	one number	784 pixels, each ink (1) or paper (0)
sum over z	2 terms	every point in a plane: infinitely many

Next three slides: generate one image this way, one step at a time.

Draw a code



$$z \sim \mathcal{N}(0, I)$$

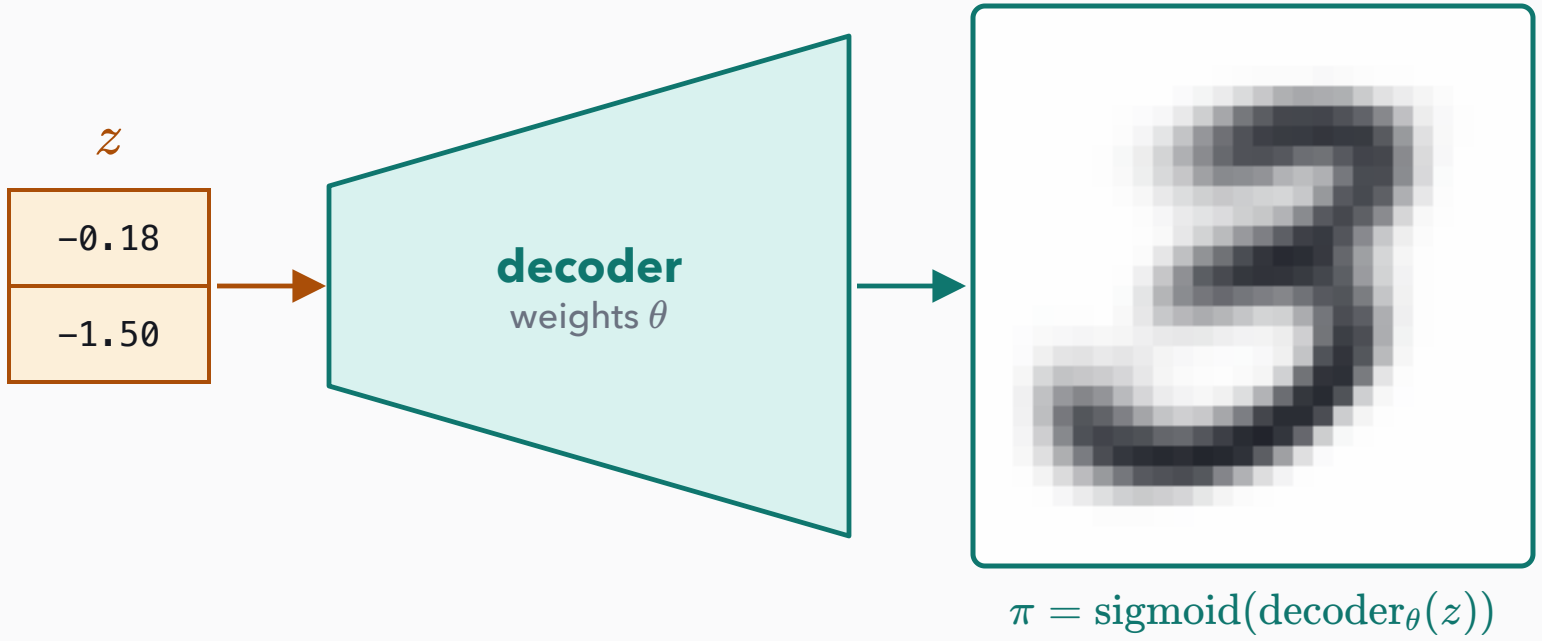
Two numbers, each from a standard bell curve: like flipping the coin, but the "coin" can land anywhere in the plane.

This draw: $z = (-0.18, \ -1.5)$.

On its own, z means nothing. The decoder will give it meaning.

GENERATING AN IMAGE · STEP 2 OF 3

Decode it into 784 pixel probabilities



each π_i is between 0 and 1

π_i = probability that pixel i is ink

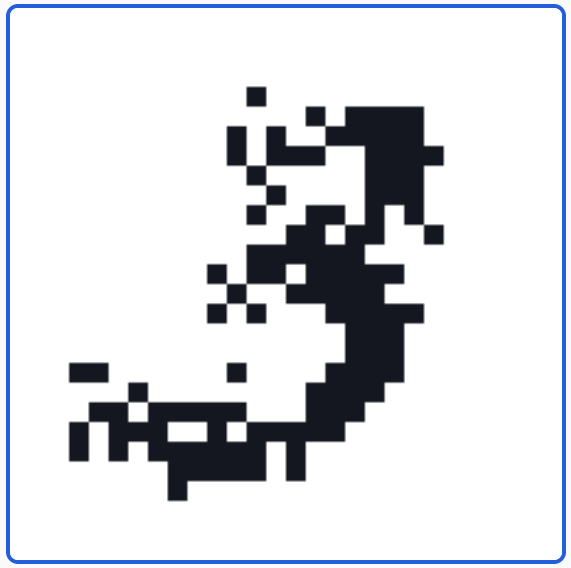
π_i is the probability that pixel i is ink. Dark grey = probably ink; white = probably paper.

GENERATING AN IMAGE · STEP 3 OF 3

Flip one coin per pixel



probabilities π



one sample x

$$x_i \sim \text{Bernoulli}(\pi_i)$$

Each pixel independently: ink with probability π_i.

That is a generated image. In practice we usually display π itself, because it looks cleaner.

THE SAME TWO STEPS AS BEFORE

Generating is easy. Learning is the catch.

Generate ✓

draw $z \sim \mathcal{N}(0, I)$, decode, flip pixels. Three slides, done.

Learn ?

choose the decoder's weights θ to make the training images probable:

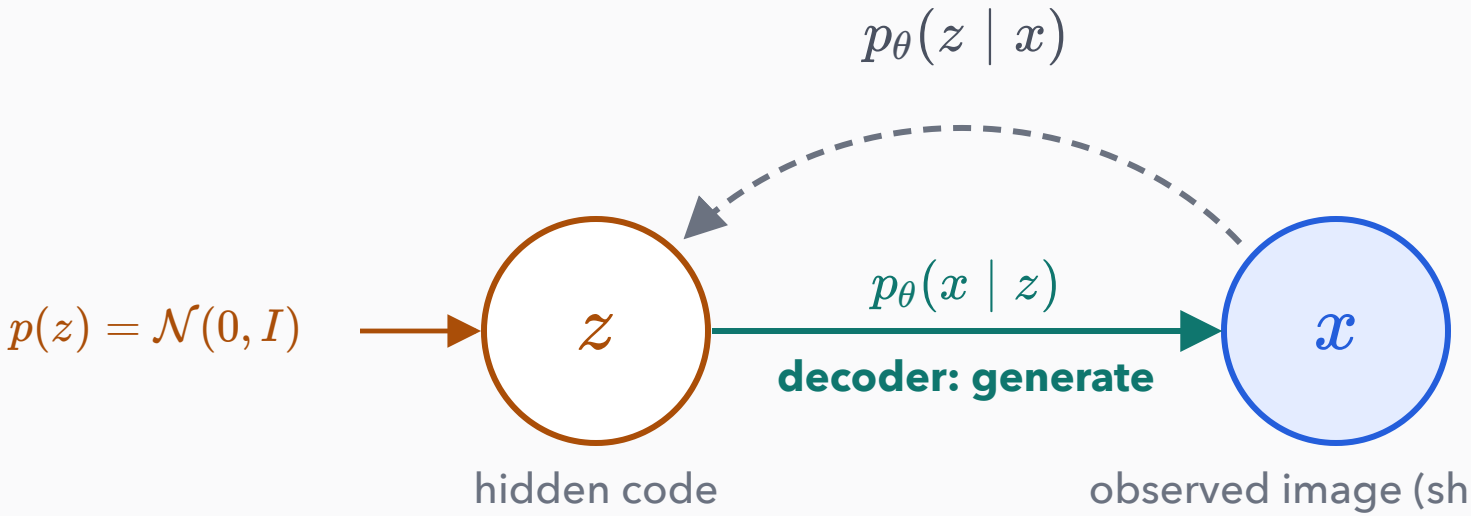
$$p_\theta(x) = \int p(z)\, p_\theta(x \mid z)\, dz$$

the two-bump sum, now over every code in the plane.

The rest of this part: why this integral is so hard, and the three ways people get around it.

THE MODEL, AS A PICTURE

Two arrows: generate down, infer up

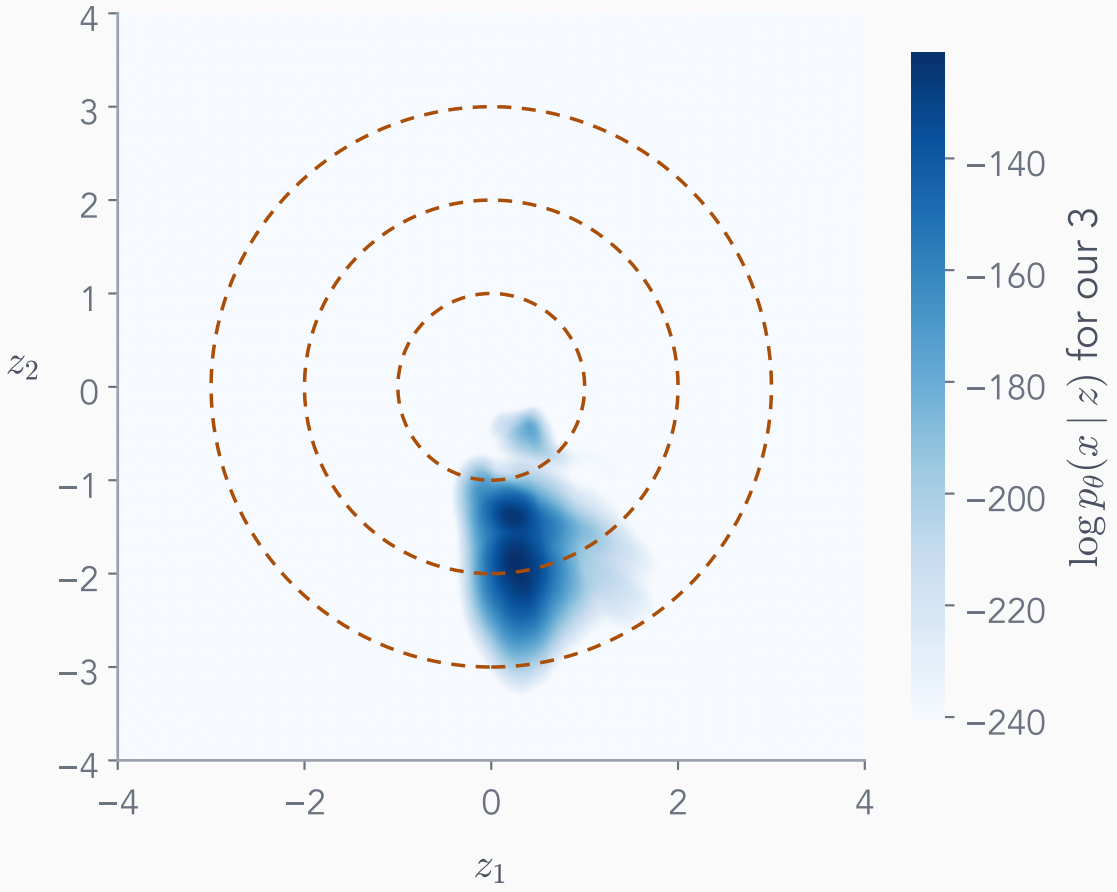


infer: which z made this x? Bayes, but it divides by $p_\theta(x)$ (dashed: out of reach)

$$p_\theta(x, z) = p(z)\, p_\theta(x \mid z)$$

MAXIMUM LIKELIHOOD FOR OUR 3

How probable is our 3 under the model?



With the coin we summed over two values:

$$p(x) = \sum_{z \in \{\mathrm{A},\mathrm{B}\}} p(z)\, p(x \mid z)$$

A code can be any point in the plane, so the sum becomes an integral:

$$p_\theta(x) = \int p(z)\; p_\theta(x \mid z)\, dz$$

2 numbers per code, so we can afford a 901 × 901 grid of decoder calls: $\log p_\theta(x) = -128.7$ nats.

WHY NOT JUST COMPUTE $p_\theta(x)$?

The integral is intractable

No formula. With two bumps, $p(x)$ was a sum of 2 terms. Here it is $\int p(z)\, p_\theta(x \mid z)\, dz$ with a neural network inside: no algebra gives the answer.

Too many terms to add up. Put a grid of 901 points on every axis of the code and add up the decoder's answers:

"Intractable" means exactly this: no formula, and far too many terms to sum.

numbers in z	decoder calls
1	901
2 (our 3)	8.1×10^5 · a second
4	6.6×10^{11} · a week
8	4.3×10^{23}
16	1.9×10^{47}

And we need $p_\theta(x)$ for every image, at every training step.

IDEA: ESTIMATE THE INTEGRAL BY SAMPLING

Random codes almost never hit the right patch

$$p_\theta(x) = \underbrace{\mathbb{E}_{z\sim p(z)}\left[\, p_\theta(x \mid z) \,\right]}_{\text{the average over codes from the prior}} \approx \frac{1}{S}\sum_{s=1}^{S} p_\theta(x \mid z_s)$$

100,000

random codes $z_s \sim \mathcal{N}(0, I)$

0.07%

land within 2 widths of our 3's patch

−363.9

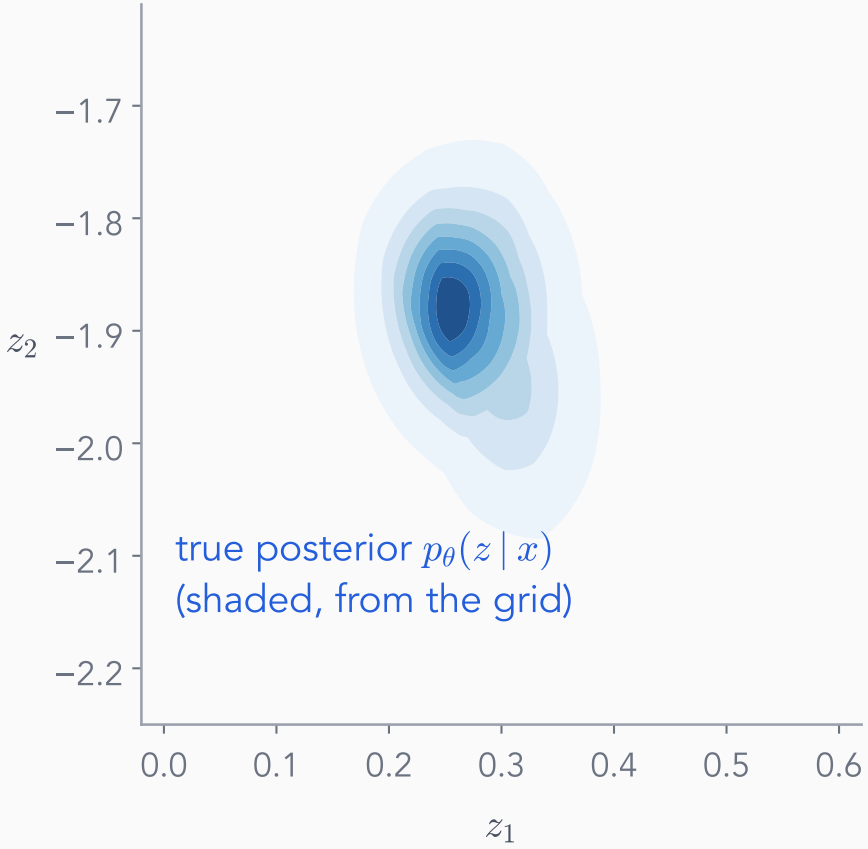
median $\log p(x \mid z)$ of a random code

−121.0

the best of the 100,000: $\log p(x \mid z)$

> **A typical random code makes our 3 about e^{243} times less probable than the best code. Blind sampling wastes almost every draw.**

The backwards step: which codes could have made our 3?



Part 1's Bayes table, with a plane of codes instead of two bumps:

$$p_\theta(z \mid x) = \frac{p(z)\, p_\theta(x \mid z)}{p_\theta(x)}$$

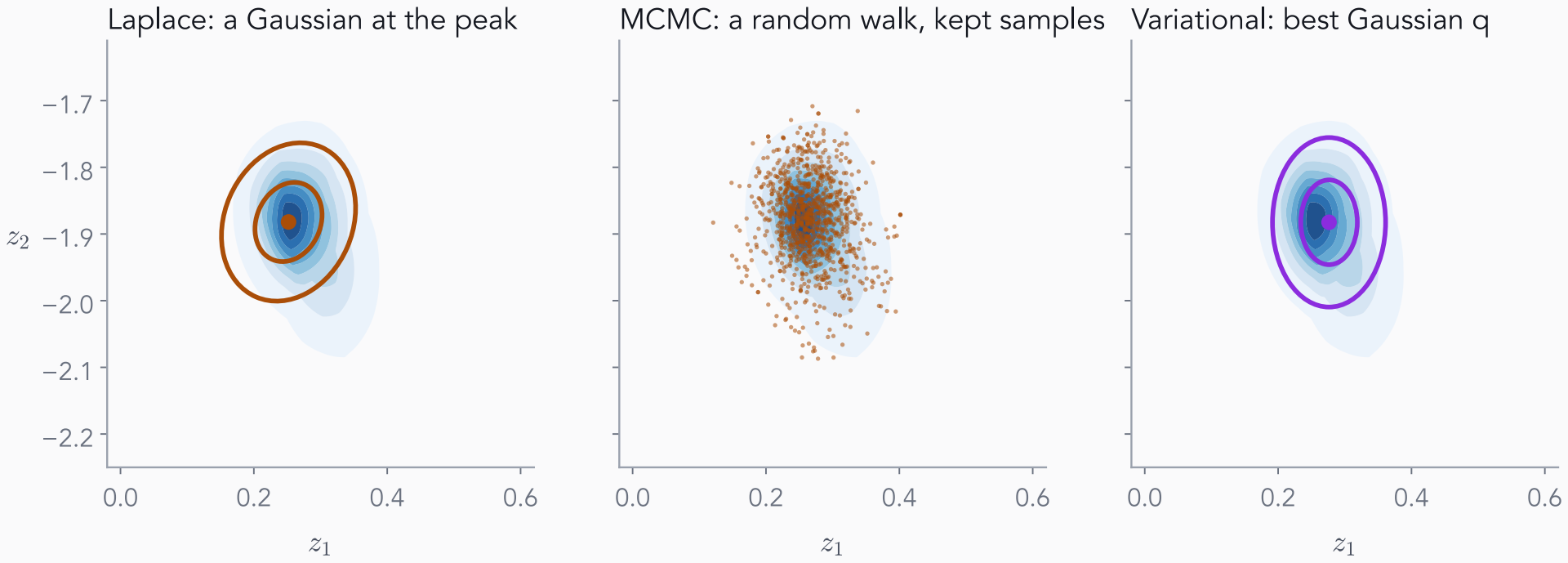
The bottom is the evidence: the same impossible integral. So the posterior is out of reach too.

From the 2-number grid we can still look at it: a small, slightly tilted patch near $(0.25,\ -1.88)$, about 0.05 wide. The prior is 1 wide: one image pins its code down tightly.

Training needs this patch for every image. Next: three ways to approximate a posterior we cannot compute.

WHEN THE POSTERIOR CANNOT BE COMPUTED, APPROXIMATE IT

Three ways, all run on our 3's real posterior



Laplace: find the peak, fit a Gaussian to the curvature there. Fast, local.

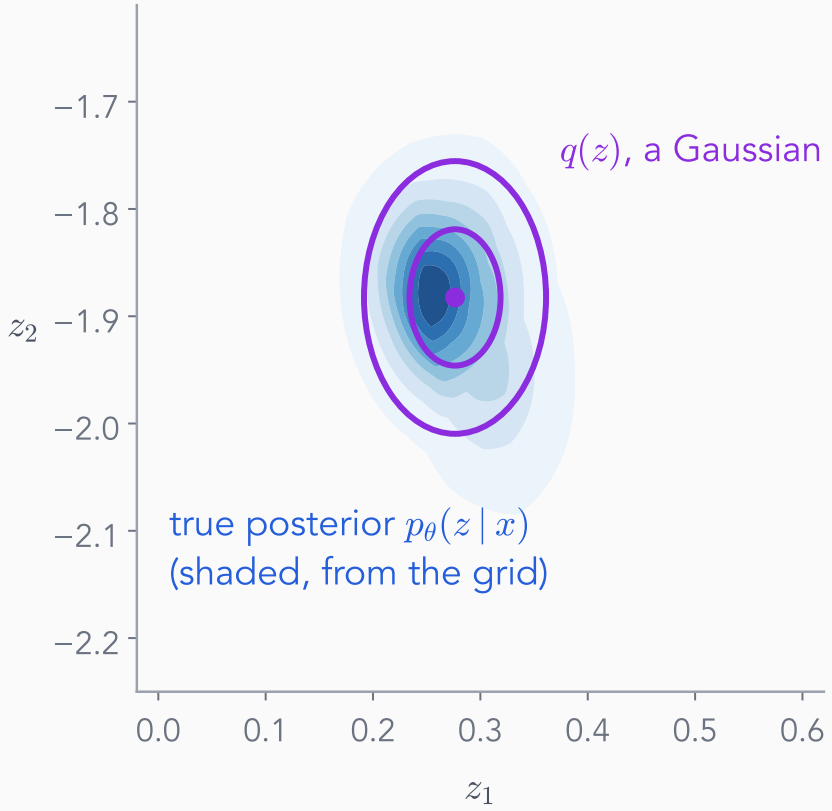
MCMC: a random walk that visits regions in proportion to the posterior. Exact in the long run, slow.

Variational: choose a simple family q and optimise it to be as close as possible. Fast, trainable.

> **Today: only the variational one. It turns inference into optimisation, which is what neural networks are good at.**

THE VARIATIONAL IDEA, FOR OUR 3

Propose a simple q, then pull it towards the true posterior



$$q(z) = \mathcal{N}\big(m,\ \mathrm{diag}(s_1^2, s_2^2)\big)$$

Four numbers to choose: a centre $m = (m_1, m_2)$ and two widths s_1, s_2.

Goal: make $q(z)$ as close as possible to $p_\theta(z \mid x)$, the shaded patch.

"As close as possible" needs a **number** for how far apart two distributions are.

That number is the KL divergence: the first thing in Session 2.

END OF PART 3 · END OF SESSION 1

What we did, and what comes next

WHAT WE DID

1. Learn = make the data probable; generate = sample. Code: one loop.
2. Autoencoders compress and rebuild (plain, denoising, sparse, contractive), but cannot generate.
3. As a probability model: prior $p(z) = \mathcal{N}(0, I)$, likelihood $p_\theta(x \mid z)$ from the decoder.
4. Training needs $p_\theta(x)$ and the posterior $p_\theta(z \mid x)$: both intractable.
5. Variational idea: a simple q, pulled close to the posterior.

SESSION 2

A distance between distributions (KL), the loss that pulls q close without ever computing $p_\theta(x)$ (the ELBO), gradients through random draws, and the VAE on MNIST.